

Relazione progetto Ingegneria del Software

September 22, 2026

Giosuè Corbioli (VR518264)

Giovanni Grigoli (VR500669)

Rey Merzari (VR517101)

Denis Modenese (VR518328)

Dip. Informatica

Università degli studi di Verona

Contenuti

1	Introduzione	3
2	Class Diagram	4
2.1	Class Diagram della modellazione concettuale	5
3	Class Diagram del software	6
3.1	Class Diagram del software	7
4	Use Case Diagram	10
5	Sequence Diagram	19
6	Sequence Diagram del software progettato	23
7	Activity Diagram	27
8	Processo di sviluppo	30
8.1	Modelli utilizzati	30
8.2	Stili Architettureali	30
8.3	Design Pattern	31
9	Testing	33
9.1	Informazioni di testing	33
9.2	Casi di testing	34
9.2.1	Unit Test	34
9.2.2	Integration Test	37
9.2.3	System Test	42

1 Introduzione

Questo documento illustra la progettazione e lo sviluppo di un sistema informatico di telemedicina dedicato alla gestione clinica dei pazienti affetti da diabete di tipo 2.

L'architettura del sistema si basa sull'interazione tra due attori principali:

I pazienti utilizzano la piattaforma per registrare autonomamente le misurazioni glicemiche giornaliere, documentare l'assunzione di farmaci e segnalare eventuali sintomi, comorbidità e terapie concomitanti.

Il medico diabetologo può, tramite la piattaforma, assegnare piani terapeutici ai suoi pazienti, oltre che compilare e consultare la loro cartella clinica e visualizzare l'andamento della glicemia.

Il sistema prevede una funzionalità di autenticazione centralizzata gestita dai responsabili del servizio.

Nelle successive sezioni sono presentati i diagrammi UML utilizzati per la progettazione e modellazione dei requisiti, seguiti da un'analisi delle decisioni strutturali e architetturali prese.

2 Class Diagram

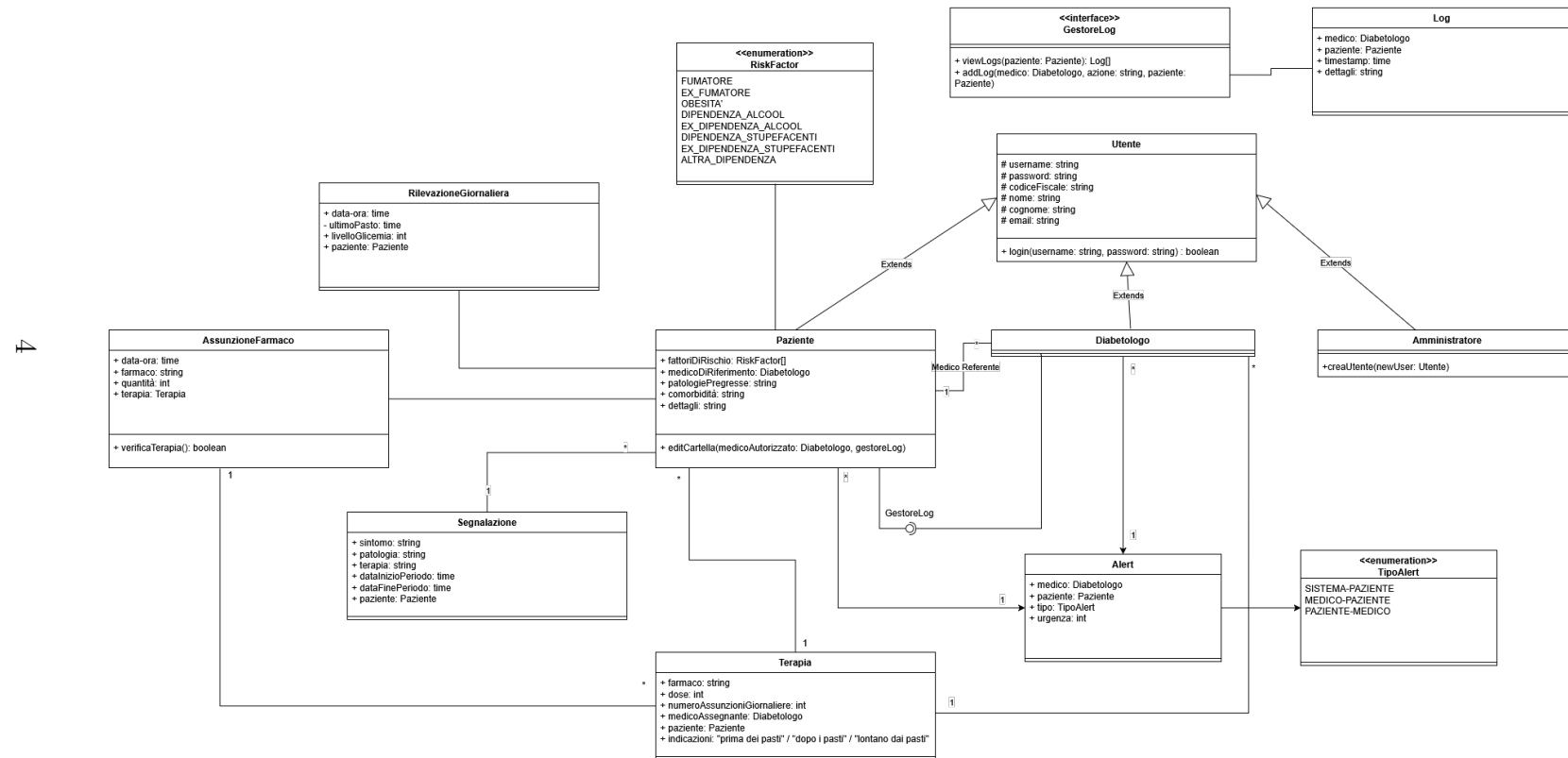


Figure 1: Class Diagram

2.1 Class Diagram della modellazione concettuale

Il sistema è incentrato sulla classe astratta *Utente*, raccoglie i dati comuni ai diversi tipi di utilizzatori, quali username, password, codice fiscale, nome, cognome ed email. Da essa derivano le classi *Paziente*, *Diabetologo* e *Responsabile*.

A ogni *Paziente* è associato un *Diabetologo* come medico di riferimento. Il *Paziente* mantiene inoltre i propri fattori di rischio, rappresentati tramite l'enumerazione *RiskFactor*, oltre che le proprie patologie pregresse, comorbidità e altri dettagli clinici. La modifica della cartella clinica (`editCartella()`) richiede l'autorizzazione di un medico e viene registrata tramite l'interfaccia *GestoreLog*, che salva ogni operazione in un oggetto *Log* (medico, paziente, timestamp, dettagli), in modo da assicurare il requisito di tracciabilità delle operazioni dei medici.

Le rilevazioni giornaliere registrano data e ora, ultimo pasto e livello di glicemia. Le assunzioni di farmaci sono verificate rispetto alla *Terapia* assegnata tramite `verificaTerapia()`.

Il paziente può inoltre effettuare *Segnalazioni* relative a sintomi, patologie o altre situazioni rilevanti. Il sistema gestisce messaggi tra paziente e diabetologo e genera messaggi di sistema destinati al paziente o al medico in presenza di condizioni che richiedono attenzione, come valori glicemici fuori soglia o mancata aderenza alla terapia.

Il *Diabetologo* può visualizzare i pazienti di propria competenza, consultare il loro andamento glicemico e gestire le relative terapie e informazioni cliniche. La classe *Terapia* contiene le informazioni relative al farmaco prescritto, alla dose, al numero di assunzioni giornaliere, al medico assegnante e alle indicazioni. Una stessa terapia può essere associata a più pazienti. Quando una terapia condivisa viene modificata per un singolo paziente, viene creata una nuova terapia per evitare di modificare quella degli altri pazienti.

Il *Responsabile* è incaricato della gestione degli utenti del sistema, in particolare della creazione e modifica dei dati relativi a pazienti e diabetologi.

La persistenza dei dati è garantita dalla classe *Database*, che centralizza il caricamento, il salvataggio, l'inserimento, la modifica e il recupero delle informazioni memorizzate nel file del database.

3 Class Diagram del software

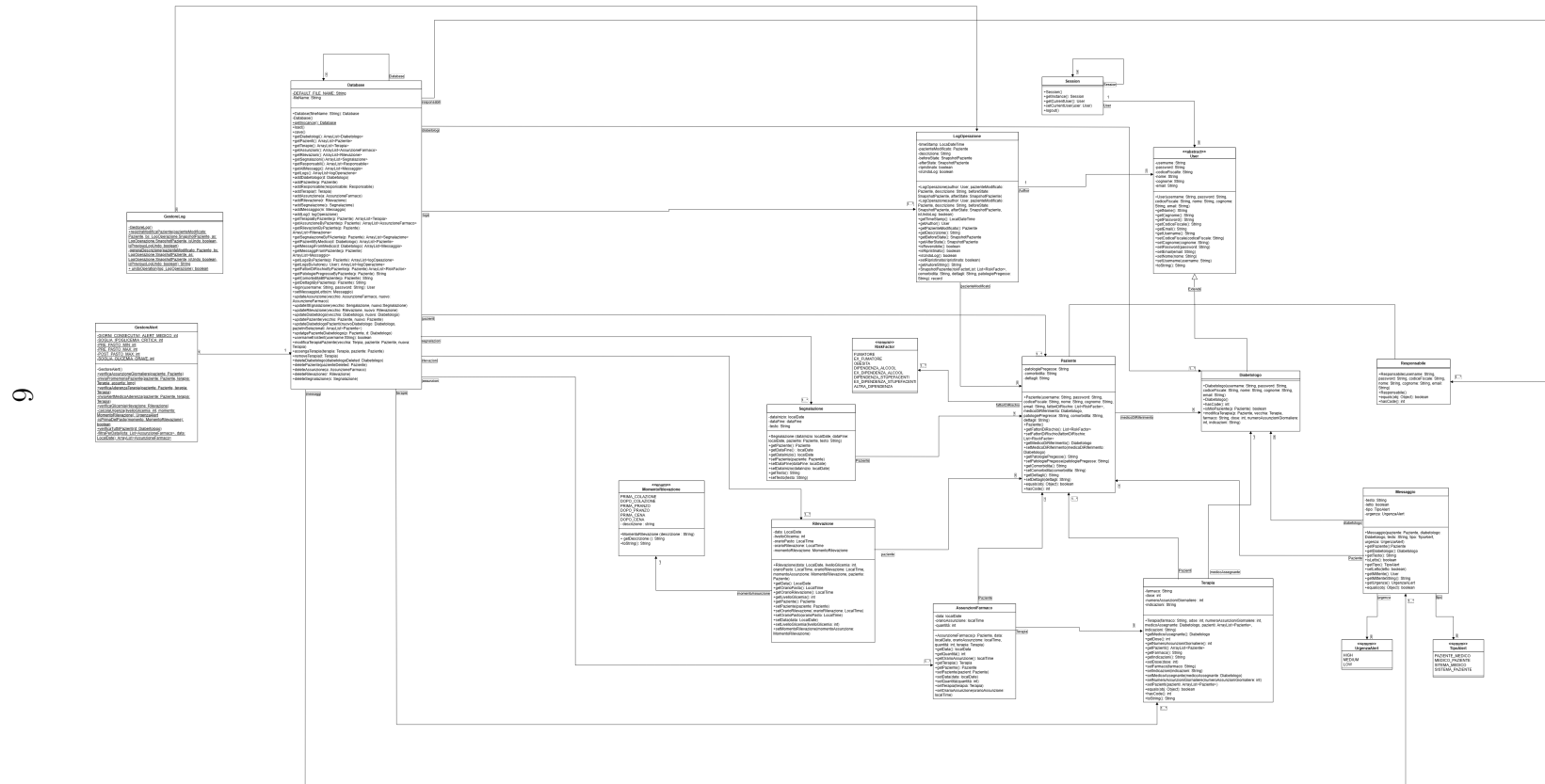


Figure 2: Class Diagram del software progettato

3.1 Class Diagram del software

Il diagramma di classi del software costituisce la traduzione del modello concettuale in una struttura di classi Java concrete, arricchita da tipi, costruttori, getter/setter e da alcune scelte implementative che nel modello concettuale non comparivano. La gerarchia portante resta invariata: una classe astratta `User` raccoglie `username`, `password`, `codice fiscale`, `nome`, `cognome` ed `email`, e da essa derivano per ereditarietà `Paziente`, `Diabetologo` e `Responsabile`. Il `Paziente` aggiunge alla superclasse gli attributi relativi a patologie pregresse, comorbidità e dettagli clinici, mentre i fattori di rischio e il medico di riferimento sono modellati come associazioni esplicite, rispettivamente verso l'enumerazione `RiskFactor` e verso `Diabetologo`. `Diabetologo` e `Responsabile` non introducono nuovi attributi rispetto a `User`, ma solo metodi propri, quali `isMioPaziente()` e `modificaTerapia()` per il primo.

Attorno a questa gerarchia ruotano le classi che modellano il dominio clinico. `Terapia` contiene `farmaco`, `dose`, `numero di assunzioni giornaliere` e `indicazioni`, ed è collegata sia al medico che l'ha assegnata sia, tramite una collezione di pazienti, a più pazienti contemporaneamente, confermando la possibilità di condividere la stessa terapia tra utenti diversi. `AssunzioneFarmaco` registra `data`, `orario` e `quantità` di ogni assunzione ed è associata sia alla terapia di riferimento sia al paziente, esponendo il metodo `verificaTerapia()`. `Rilevazione` tiene traccia della glicemia rilevata e degli orari di pasto e rilevazione, associata all'enumerazione `MomentoRilevazione`, la quale non si limita a elencare i possibili momenti della giornata ma incorpora anche una descrizione testuale associata a ciascun valore. `Segnalazione` si compone infine di un testo libero delimitato da un periodo di inizio e fine, sempre riferita al paziente che l'ha effettuata.

La classe `Messaggio` unifica due concetti tenuti distinti nel modello concettuale, ovvero le comunicazioni dirette tra paziente e medico e gli avvisi generati automaticamente dal sistema: un campo dedicato al tipo distingue infatti le comunicazioni paziente-medico, medico-paziente e quelle generate dal sistema verso l'uno o l'altro, mentre un secondo campo ne definisce il livello di urgenza, evitando così la necessità di due classi separate.

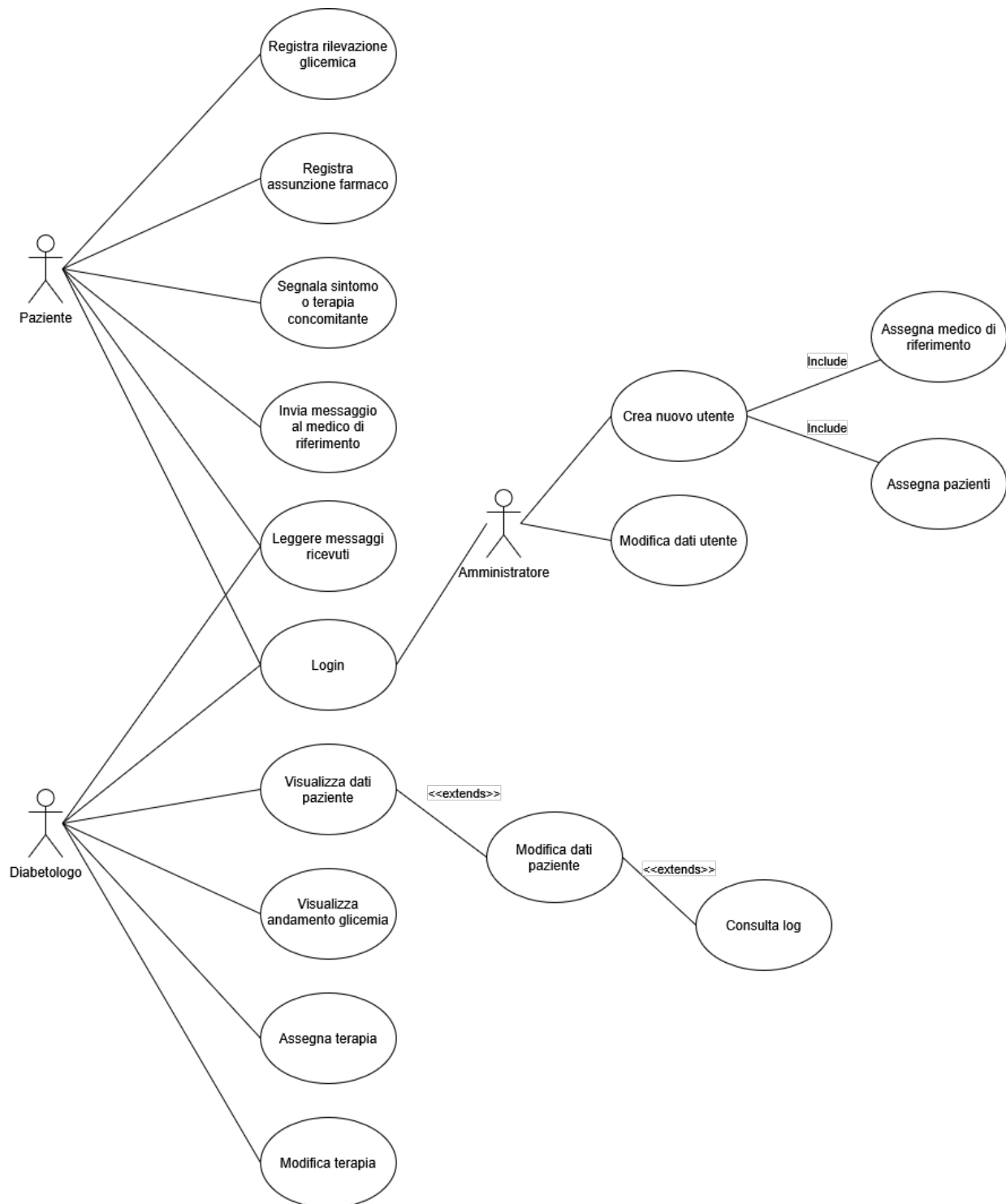
Il diagramma software introduce inoltre alcune classi infrastrutturali assenti nel modello concettuale, necessarie a gestire la logica applicativa. La classe `Database` è realizzata secondo il pattern `Singleton` e centralizza non solo tutte le collezioni di dati del sistema, ma anche gran parte della logica di business, dalle operazioni di autenticazione alla creazione, modifica e cancellazione di ogni entità. Accanto a essa, la classe `Session`, anch'essa `Singleton`, gestisce separatamente l'utente attualmente autenticato. La classe `GestoreAlert` implementa la logica di generazione degli avvisi clinici, definendo soglie config-

urabili relative a livelli critici di glicemia e a fasce orarie rispetto ai pasti, e verificando periodicamente l'aderenza alla terapia e i valori fuori soglia, generando di conseguenza i messaggi di sistema opportuni.

Merita infine una menzione particolare la classe `GestoreLog` che, a differenza della sua controparte nel modello concettuale, dove è definita come interfaccia, viene qui implementata come classe concreta che non si limita a registrare le modifiche apportate alla cartella clinica, ma ne consente anche l'annullamento. Questa funzionalità è resa possibile dalla classe `LogOperazione`, la quale conserva, oltre a medico, paziente, timestamp e dettagli dell'operazione, anche lo stato del paziente immediatamente precedente e successivo alla modifica, realizzando di fatto un meccanismo di tipo memento a supporto della tracciabilità e della reversibilità delle operazioni cliniche.

Non è stato rappresentato il diagramma delle classi relative ai controller, perché la loro elevata complessità e il numero di attributi e metodi avrebbero reso il diagramma eccessivamente dettagliato, compromettendone la leggibilità e la funzionalità. Le varie classi dei controller svolgono principalmente il ruolo di collegamento tra l'interfaccia grafica e la logica dell'applicazione, per una consultazione più approfondita è possibile fare riferimento direttamente al codice sorgente del progetto.

4 Use Case Diagram



ID	1
Nome	Login
Attori	Paziente, Diabetologo, Amministratore
Descrizione	Permette agli utenti di entrare nel sistema
Precondizioni	L'utente deve essere registrato
Sequenza principale	1. L'utente inserisce email e password 2. L'utente clicca sul pulsante login 3. L'utente entra nel sistema
Sequenza alternativa	Se le credenziali non sono valide, mostra un errore
Postcondizioni	L'utente è autenticato e viene associato a una sessione attiva.

ID	2
Nome	RegistrazioneRilevazioneGlicemica
Attori	Paziente
Descrizione	Il paziente registra il proprio livello di glicemia
Precondizioni	Il paziente ha effettuato il login
Sequenza principale	1. Il paziente clicca sul pulsante "aggiungi" nella sezione Rilevazioni Giornaliere 2. Il sistema chiede il livello di glicemia, la data, l'ora, il momento della giornata e l'orario dell'ultimo pasto 3. Il paziente inserisce i dati richiesti 4. Il paziente clicca sul pulsante "salva" 5. Il sistema salva la rilevazione nella cartella clinica del paziente
Sequenza alternativa	Se il valore è fuori dalla soglia attesa, il sistema genera una segnalazione al diabetologo di riferimento. Il sistema determina il livello di gravità in base allo scostamento dalla soglia e notifica il diabetologo con modalità corrispondente alla gravità
Postcondizioni	La rilevazione glicemica è salvata e associata al paziente; se il valore glicemico è fuori soglia, viene generata una segnalazione al diabetologo

ID	3
Nome	RegistrazioneAssunzioneFarmaco
Attori	Paziente
Descrizione	Il paziente registra l'assunzione di un farmaco
Precondizioni	Il paziente ha effettuato il login; al paziente è stata assegnata almeno una terapia dal diabetologo
Sequenza principale	1. Il paziente clicca sul pulsante "aggiungi" nella sezione Assunzione Farmaco 2. Il sistema chiede la data, l'ora, la quantità di farmaco assunta e la terapia per la quale lo si sta assumendo 3. Il paziente inserisce i dati richiesti 4. Il paziente clicca sul pulsante "salva" 5. Il sistema salva la rilevazione nella cartella clinica del paziente
Sequenza alternativa	Se l'assunzione non è coerente con la terapia prescritta, il sistema lo segnala al paziente
Postcondizioni	L'assunzione è salvata e associata al paziente

ID	4
Nome	Segnalazione
Attori	Paziente
Descrizione	Il paziente segnala un sintomo o una terapia concomitante
Precondizioni	Il paziente ha effettuato il login
Sequenza principale	1. Il paziente clicca sul pulsante "aggiungi" nella sezione Segnalazioni 2. Il sistema chiede la data di inizio della terapia o del sintomo, opzionalmente la data di fine, e una descrizione 3. Il paziente inserisce i dati richiesti 4. Il paziente clicca sul pulsante "salva" 5. Il sistema salva la segnalazione nella cartella clinica del paziente
Sequenza alternativa	Nessuna
Postcondizioni	La segnalazione è salvata e associata al paziente

ID	5
Nome	InvioMessaggioAlMedico
Attori	Paziente
Descrizione	Il paziente invia un messaggio (email) al proprio medico di riferimento
Precondizioni	Il paziente ha effettuato il login; al paziente è associato un medico di riferimento
Sequenza principale	1. Il paziente clicca sul pulsante "Scrivi E-Mail" 2. Il paziente inserisce il testo della mail 3. Il paziente clicca sul pulsante "Invia" 4. Il sistema invia il messaggio al diabetologo
Sequenza alternativa	Nessuno
Postcondizioni	Il messaggio è recapitato al diabetologo di riferimento

ID	6
Nome	LetturaMessaggioRicevuto
Attori	Paziente, Diabetologo
Descrizione	L'utente legge i messaggi ricevuti
Precondizioni	L'utente ha effettuato il login
Sequenza principale	1. L'utente clicca sul pulsante raffigurante un messaggio 2. Il sistema mostra l'elenco dei messaggi ricevuti 3. L'utente clicca sul pulsante "Apri" del messaggio che vuole leggere 4. Il sistema mostra il contenuto del messaggio
Sequenza alternativa	Se non è presente nessun messaggio, il sistema mostra una lista vuota
Postcondizioni	I messaggi visionati sono contrassegnati come letti

ID	7
Nome	VisualizzazioneDatiPaziente
Attori	Diabetologo
Descrizione	Il diabetologo consulta i dati di un paziente: fattori di rischio, patologie pregresse, comorbidità e dettagli
Precondizioni	Il diabetologo ha effettuato il login; il diabetologo è il medico di riferimento di almeno un paziente
Sequenza principale	1. Il diabetologo clicca sul pulsante "Info Paziente" per un paziente nell'elenco 2. Il sistema mostra la cartella clinica contenente i dati del paziente selezionato: fattori di rischio, patologie pregresse, comorbidità e dettagli 3. Il diabetologo consulta i dati 4. Extension point (ModificaDatiPaziente)
Sequenza alternativa	Nessuno
Postcondizioni	Nessuna

ID	8
Nome	ModificaDatiPaziente (Estensione)
Attori	Diabetologo
Descrizione	Il diabetologo modifica i dati di un paziente: fattori di rischio, patologie pregresse, comorbidità e dettagli
Precondizioni	Il diabetologo ha effettuato il login; il diabetologo è il medico di riferimento di almeno un paziente
Sequenza principale	1. Il diabetologo modifica i dati del paziente da aggiornare: fattori di rischio, patologie pregresse, comorbidità o dettagli 2. Il diabetologo clicca sul pulsante "Salva" 3. Il sistema salva i dati modificati 4. Extension point (ConsultazioneLog)
Sequenza alternativa	Nessuna
Postcondizioni	I dati del paziente aggiornati vengono salvati

ID	9
Nome	ConsultazioneLog (Estensione)
Attori	Diabetologo
Descrizione	Il diabetologo consulta il registro delle modifiche dei dati dei pazienti effettuate
Precondizioni	Il diabetologo ha effettuato il login
Sequenza principale	1. Il diabetologo clicca sul pulsante "Apri storico" 2. Il diabetologo consulta i dati
Sequenza alternativa	Se non è stata effettuata nessuna operazione, il sistema mostra una lista vuota; il diabetologo può ripristinare i dati di un paziente salvati in precedenza
Postcondizioni	Nessuna

ID	10
Nome	VisualizzazioneAndamentoGlicemia
Attori	Diabetologo
Descrizione	Il diabetologo visualizza, tramite un grafico, l'andamento della glicemia di un paziente: giorno per giorno, settimana per settimana o mese per mese
Precondizioni	Il diabetologo ha effettuato il login; il diabetologo è il medico di riferimento di almeno un paziente
Sequenza principale	1. Il diabetologo clicca sul pulsante "Andamento" per un paziente nell'elenco 2. Il sistema visualizza i dati sull'andamento glicemico 3. Il diabetologo consulta i dati
Sequenza alternativa	Nessuna
Postcondizioni	Nessuna

ID	11
Nome	AssegnazioneTerapia
Attori	Diabetologo
Descrizione	Il diabetologo assegna una terapia a un suo paziente
Precondizioni	Il diabetologo ha effettuato il login; il paziente è registrato nel sistema e assegnato al diabetologo
Sequenza principale	1. Il diabetologo clicca sul pulsante "Terapia" per un paziente nell'elenco 2. Il diabetologo clicca sul pulsante "Aggiungi" 3. Il diabetologo clicca sul pulsante "Nuova" 4. Il sistema chiede il farmaco, il numero di assunzioni giornaliere, la quantità per assunzione ed eventuali indicazioni aggiuntive 5. Il diabetologo inserisce i dati richiesti 6. Il diabetologo clicca sul pulsante "Salva"
Sequenza alternativa	Il diabetologo clicca sul pulsante "Esistente" e assegna una terapia già assegnata in passato
Postcondizioni	La nuova terapia è salvata, attiva e visualizzabile dal paziente; la terapia viene salvata dal sistema

ID	12
Nome	ModificaTerapia
Attori	Diabetologo
Descrizione	Il diabetologo modifica la terapia esistente di un suo paziente
Precondizioni	Il diabetologo ha effettuato il login; al paziente è già associata una terapia
Sequenza principale	1. Il diabetologo clicca sul pulsante "Terapia" per un paziente nell'elenco 2. Il diabetologo clicca sul pulsante "Modifica" relativo a una terapia 3. Il sistema permette di modificare il farmaco, il numero di assunzioni giornaliere, la quantità per assunzione ed eventuali indicazioni aggiuntive 4. Il diabetologo modifica i dati necessari 5. Il diabetologo clicca sul pulsante "Salva" 6. Il sistema aggiorna la terapia
Sequenza alternativa	Se il diabetologo non modifica nessun dato, la terapia non viene aggiornata
Postcondizioni	La terapia è attiva e visualizzabile dal paziente; è salvata dal sistema se nuova

ID	13
Nome	CreazioneNuovoUtente
Attori	Amministratore
Descrizione	L'amministratore crea un nuovo account utente (paziente o diabetologo) nel sistema
Precondizioni	L'amministratore ha effettuato il login
Sequenza principale	1. L'amministratore clicca sul pulsante "Aggiungi medici"/"Aggiungi pazienti" 2. Il sistema chiede di inserire nome, cognome, codice fiscale, email, username e password 3. L'amministratore inserisce i dati 4. L'amministratore clicca sul pulsante "Conferma"
Sequenza alternativa	Se esiste un altro utente con lo stesso username, viene mostrato un messaggio di errore e si ritorna al passo 3
Postcondizioni	Il nuovo utente viene creato ed è abilitato all'accesso

ID	13a
Nome	AssegnaMedicoDiRiferimento
Attori	Amministratore
Descrizione	L'amministratore associa al paziente un diabetologo come medico di riferimento
Precondizioni	È in corso la creazione o modifica di un account paziente
Sequenza principale	1. Include (CreazioneNuovoUtente) 2. Il sistema mostra l'elenco dei diabetologi 3. L'amministratore seleziona il diabetologo di riferimento
Sequenza alternativa	Nessuna
Postcondizioni	Il paziente viene associato al diabetologo selezionato

ID	13b
Nome	AssegnaPazienti
Attori	Amministratore
Descrizione	L'amministratore associa uno o più pazienti a un diabetologo
Precondizioni	È in corso la creazione o modifica di un account diabetologo
Sequenza principale	1. Include (CreazioneNuovoUtente) 2. Il sistema mostra l'elenco dei pazienti 3. L'amministratore seleziona uno o più pazienti da assegnare
Sequenza alternativa	Se non esiste nessun paziente, il sistema mostra una lista vuota; se non viene selezionato alcun paziente, il medico viene salvato senza pazienti associati
Postcondizioni	I pazienti vengono associati al diabetologo selezionato

ID	14
Nome	ModificaDatiUtente
Attori	Amministratore
Descrizione	L'amministratore modifica i dati anagrafici o di accesso di un utente esistente
Precondizioni	L'amministratore ha effettuato il login; l'utente da modificare esiste nel sistema
Sequenza principale	1. L'amministratore clicca sul pulsante "Cambia credenziali" 2. L'amministratore modifica le credenziali del profilo selezionato
Sequenza alternativa	Se non esiste nessun utente, il sistema mostra una lista vuota; se esiste un altro utente con lo stesso username, viene mostrato un messaggio di errore e si ritorna al punto 2
Postcondizioni	I dati dell'utente vengono aggiornati vengono salvati

5 Sequence Diagram

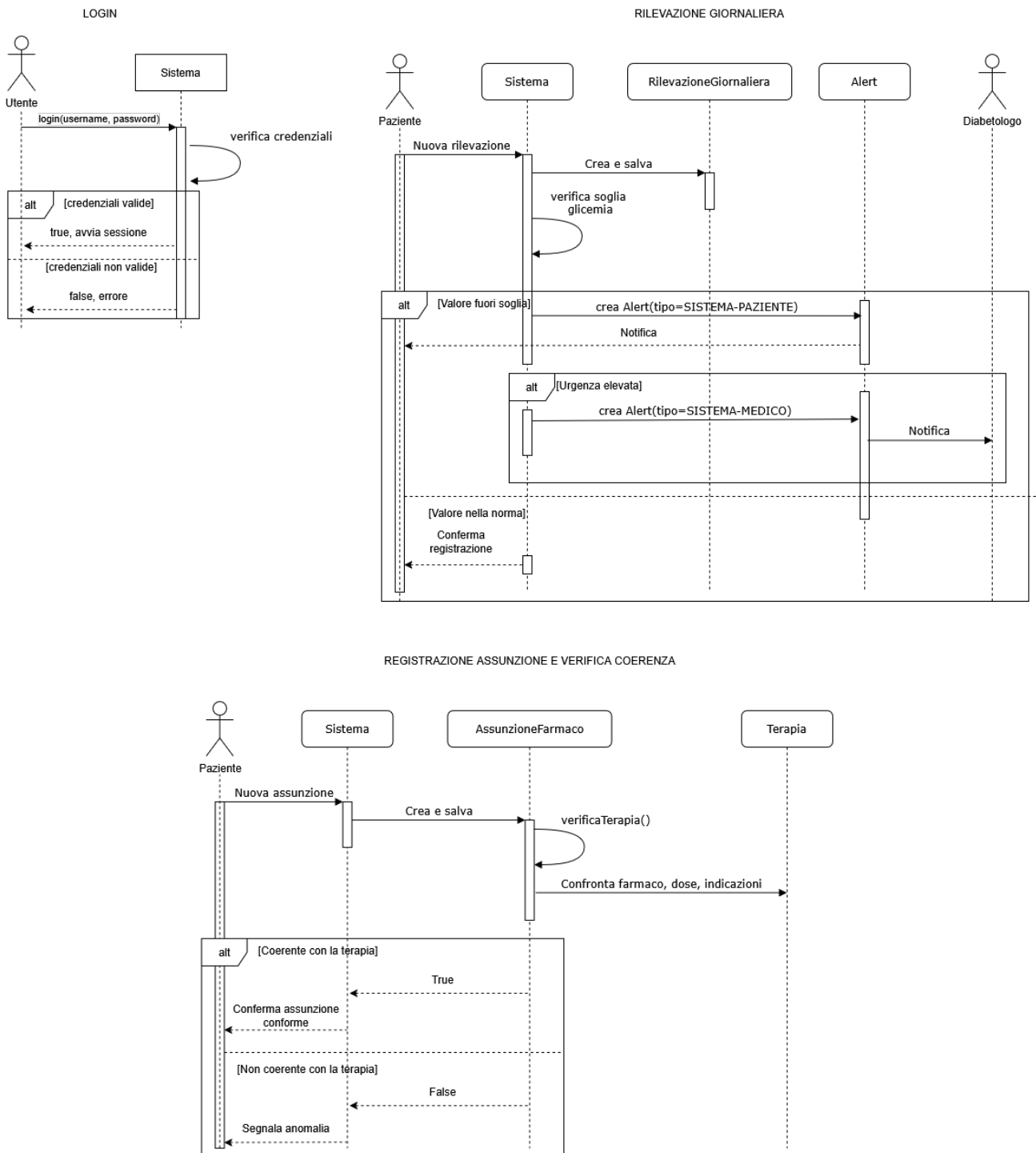


Figure 3: Sequence Diagram (Parte 1)

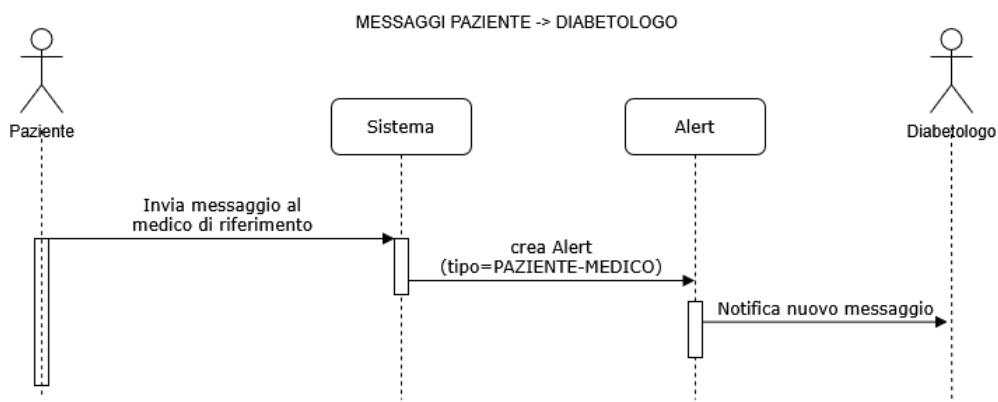
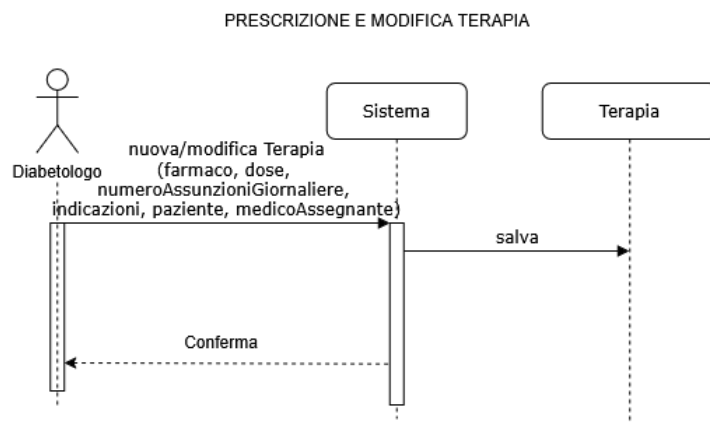
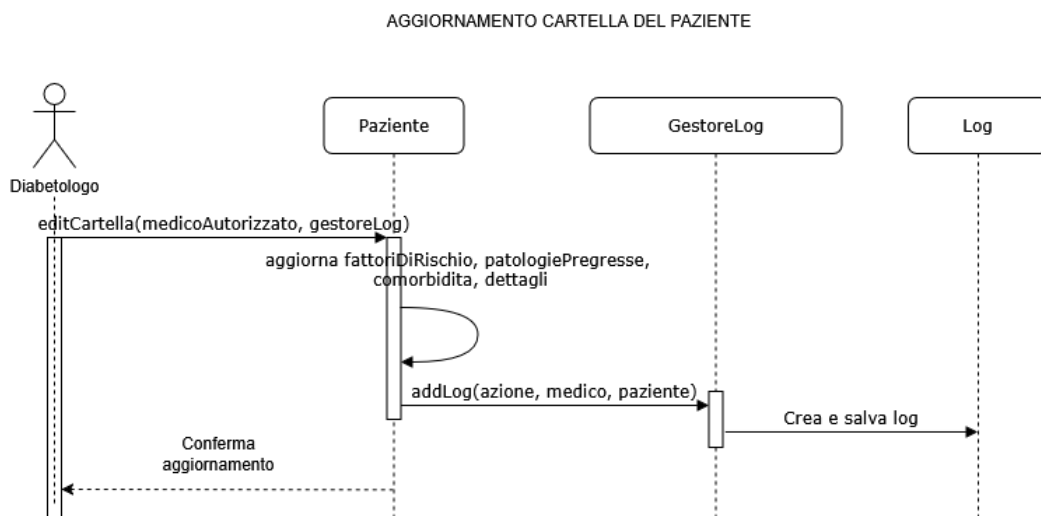


Figure 4: Sequence Diagram (Parte 2)

Login: L'utente inserisce username e password nella schermata di accesso. Il sistema inoltra le credenziali al database, il quale verifica la corrispondenza con un utente registrato. Se sono valide, viene avviata la sessione ed è aperta la schermata corrispondente al ruolo dell'utente. In caso contrario il sistema segnala un errore e impedisce l'accesso.

Rilevazione giornaliera: il paziente inserisce i dati relativi a una nuova rilevazione, ovvero data, livello di glicemia, orario, orario dell'ultimo pasto e momento della rilevazione. Dopodiché il sistema la salva e verifica il livello di glicemia riportato rispetto alle soglie previste. Se esso è fuori dai limiti, viene creato un alert che notifica il diabetologo di riferimento di tale irregolarità, con un livello di urgenza maggiore se la glicemia si discosta particolarmente dalla norma. Il sistema conferma infine l'avvenuta registrazione all'utente.

Registrazione di un'assunzione e verifica della coerenza: il paziente inserisce i dati relativi all'assunzione di un farmaco, indicando data, orario, quantità e terapia di riferimento. Tali dati vengono salvati e verificati, confrontandoli con le informazioni della terapia prescritta dal medico. Se essi sono discrepanti con la terapia, ciò viene segnalato all'utente.

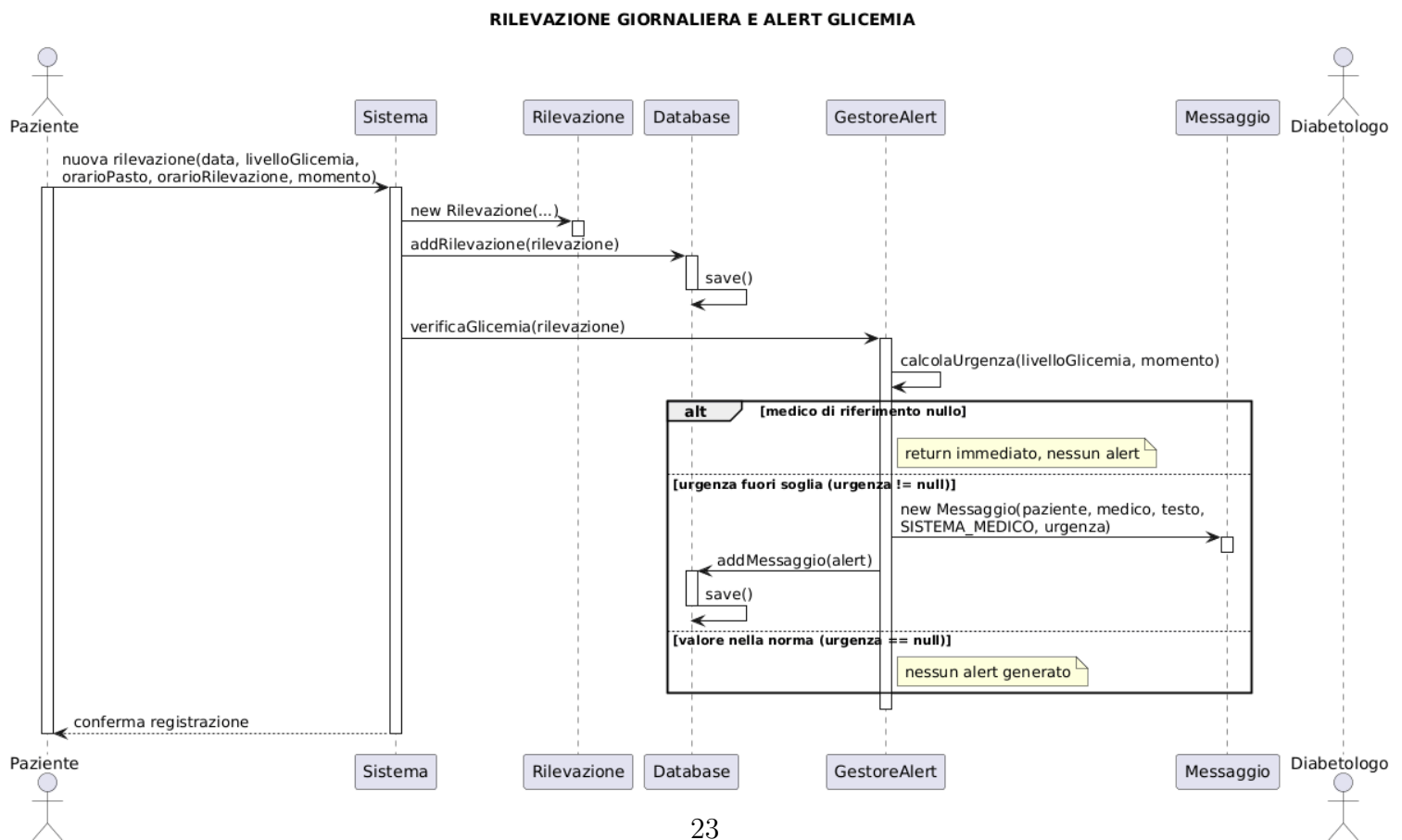
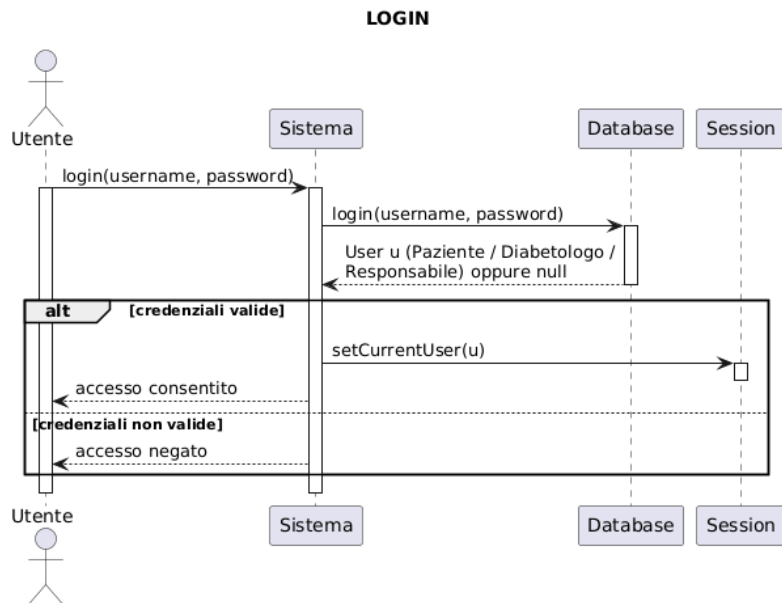
Aggiornamento cartella del paziente: il diabetologo di riferimento accede alle informazioni del paziente e ne modifica i dati, ovvero fattori di rischio, patologie pregresse, comorbidità e dettagli aggiuntivi. I dati vengono salvati e viene creato un log dell'operazione, associando la modifica all'utente che l'ha effettuata, in modo da mantenere la tracciabilità delle modifiche apportate.

Prescrizione e modifica di una terapia: il diabetologo di riferimento crea una nuova terapia oppure modifica le specifiche di una terapia già assegnata in passato modificando i dati rilevanti. Nel caso di modifica di una terapia condivisa da più pazienti, il sistema crea una nuova versione della terapia per il paziente interessato, evitando di modificare la terapia degli altri pazienti che continuano a utilizzare quella precedente.

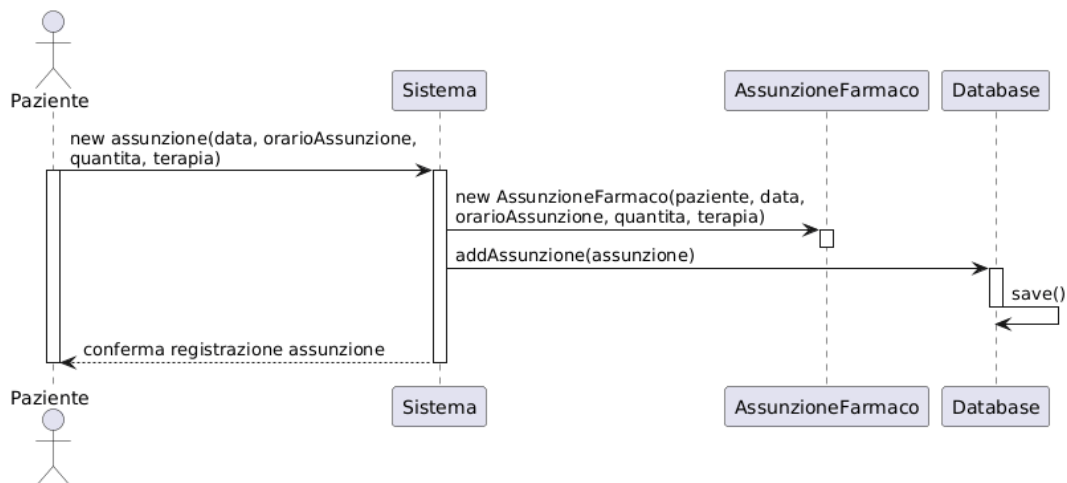
Scambio di messaggi tra paziente e diabetologo: il paziente apre la schermata di comunicazione e scrive un messaggio al proprio medico di riferimento. Il medico viene notificato e può aprirlo, leggerlo e inviare una risposta al paziente.

TODO: Lasciamo "inviare una risposta al paziente"?

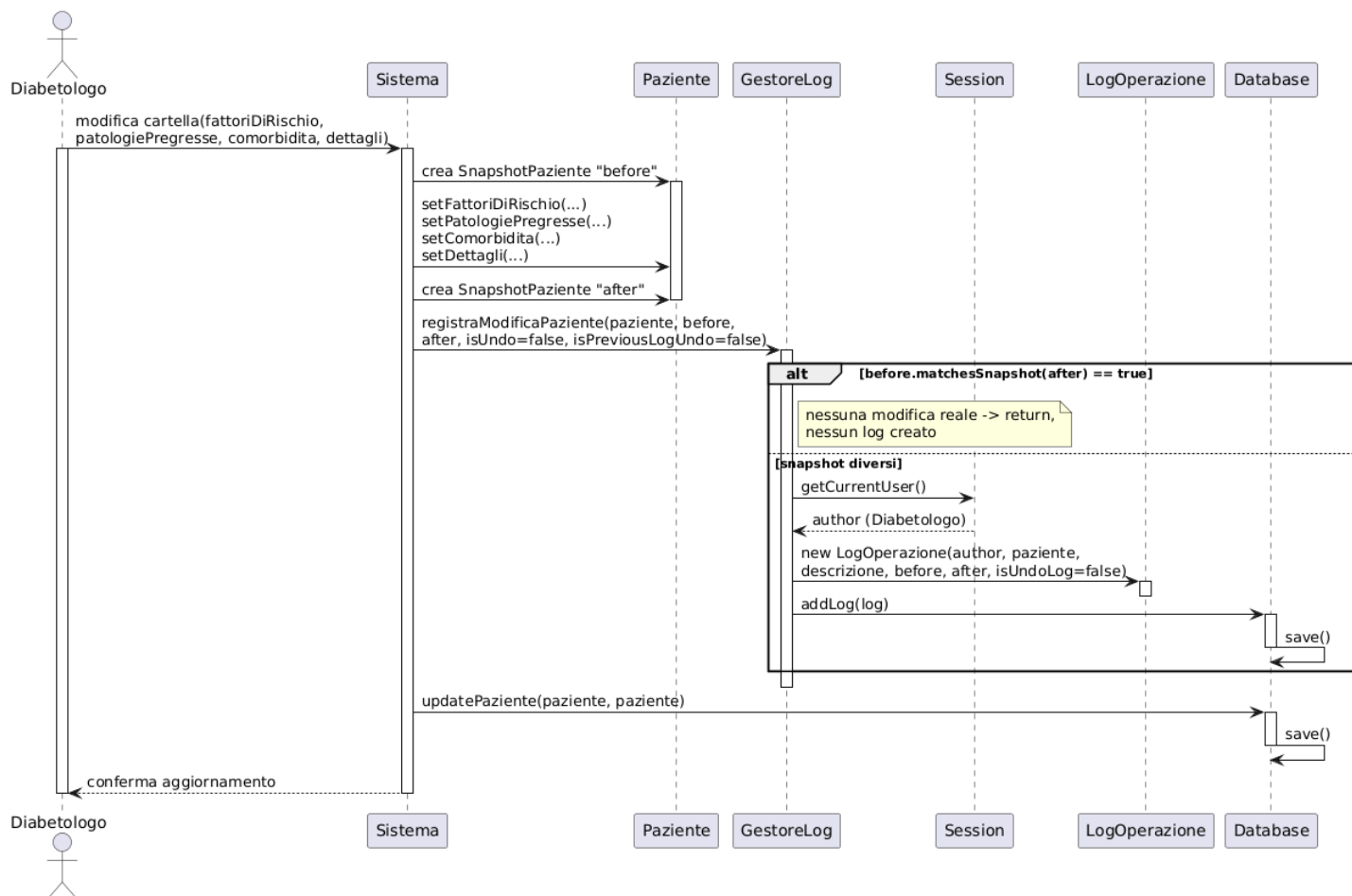
6 Sequence Diagram del software progettato



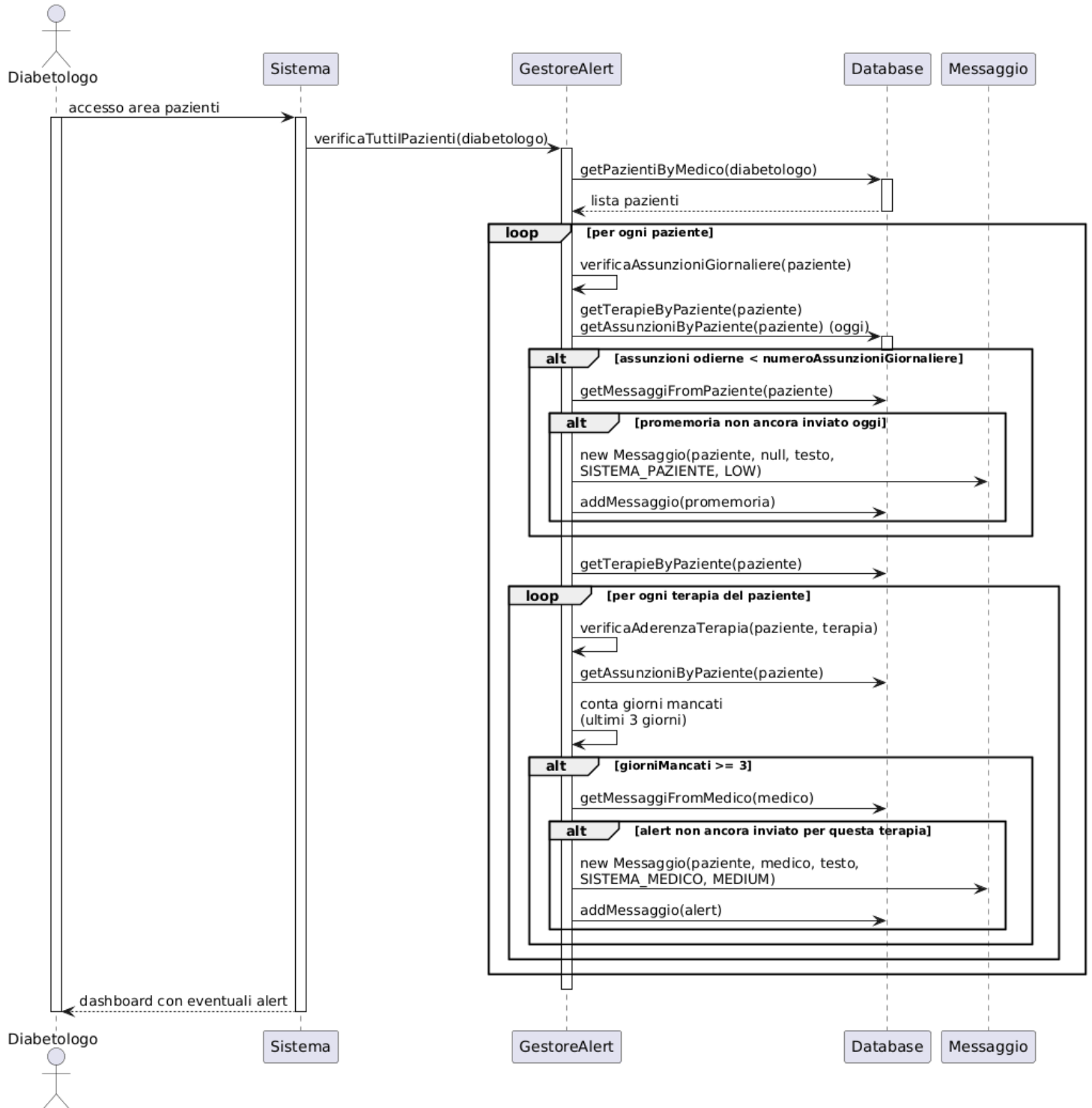
REGISTRAZIONE ASSUNZIONE FARMACO



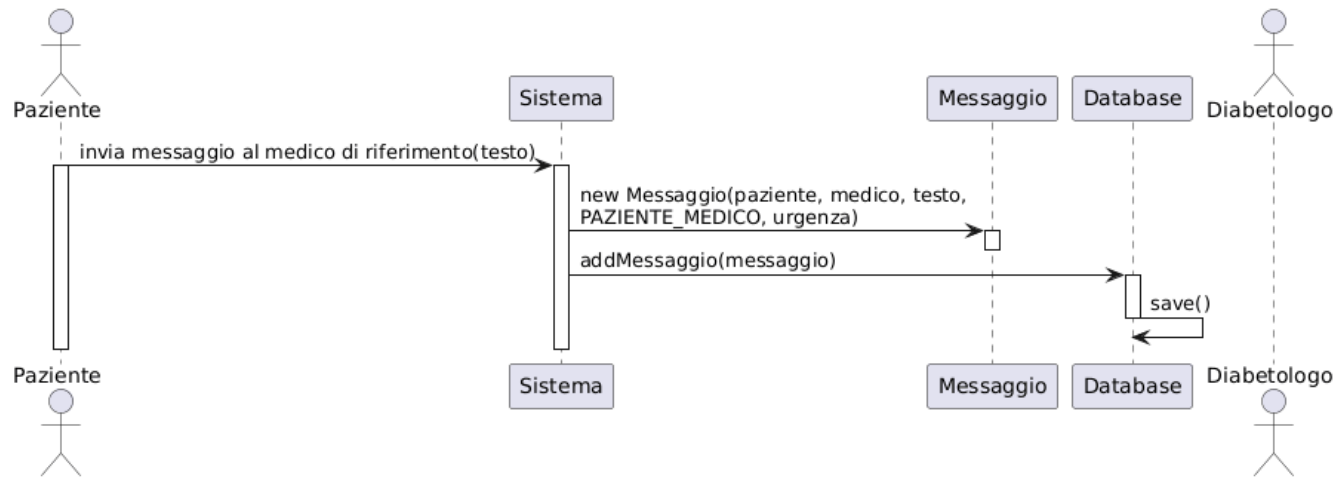
AGGIORNAMENTO CARTELLA PAZIENTE



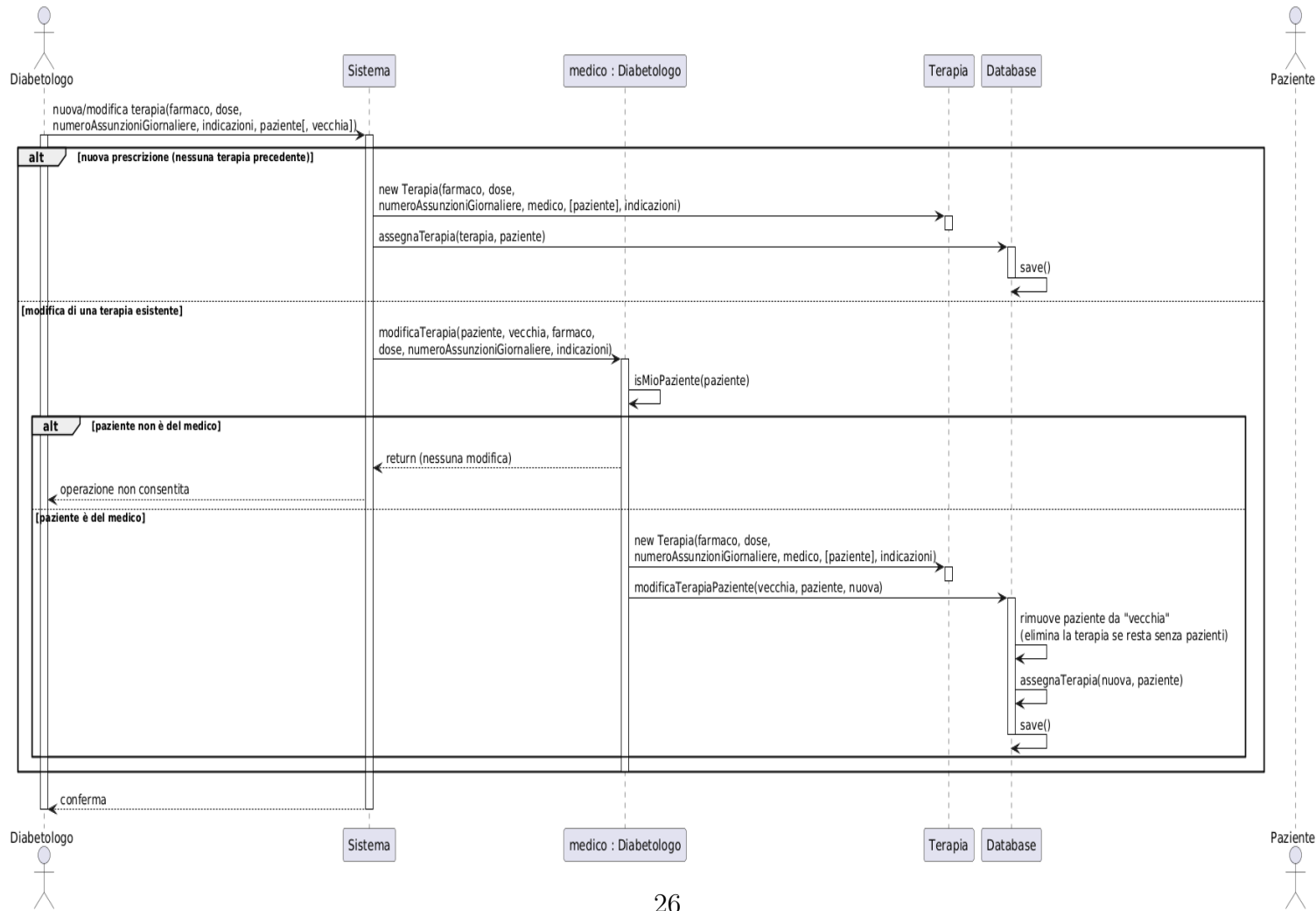
CONTROLLO AUTOMATICO ADERENZA E PROMEMORIA



MESSAGGIO PAZIENTE - DIABETOLOGO



PRESCRIZIONE E MODIFICA TERAPIA



7 Activity Diagram

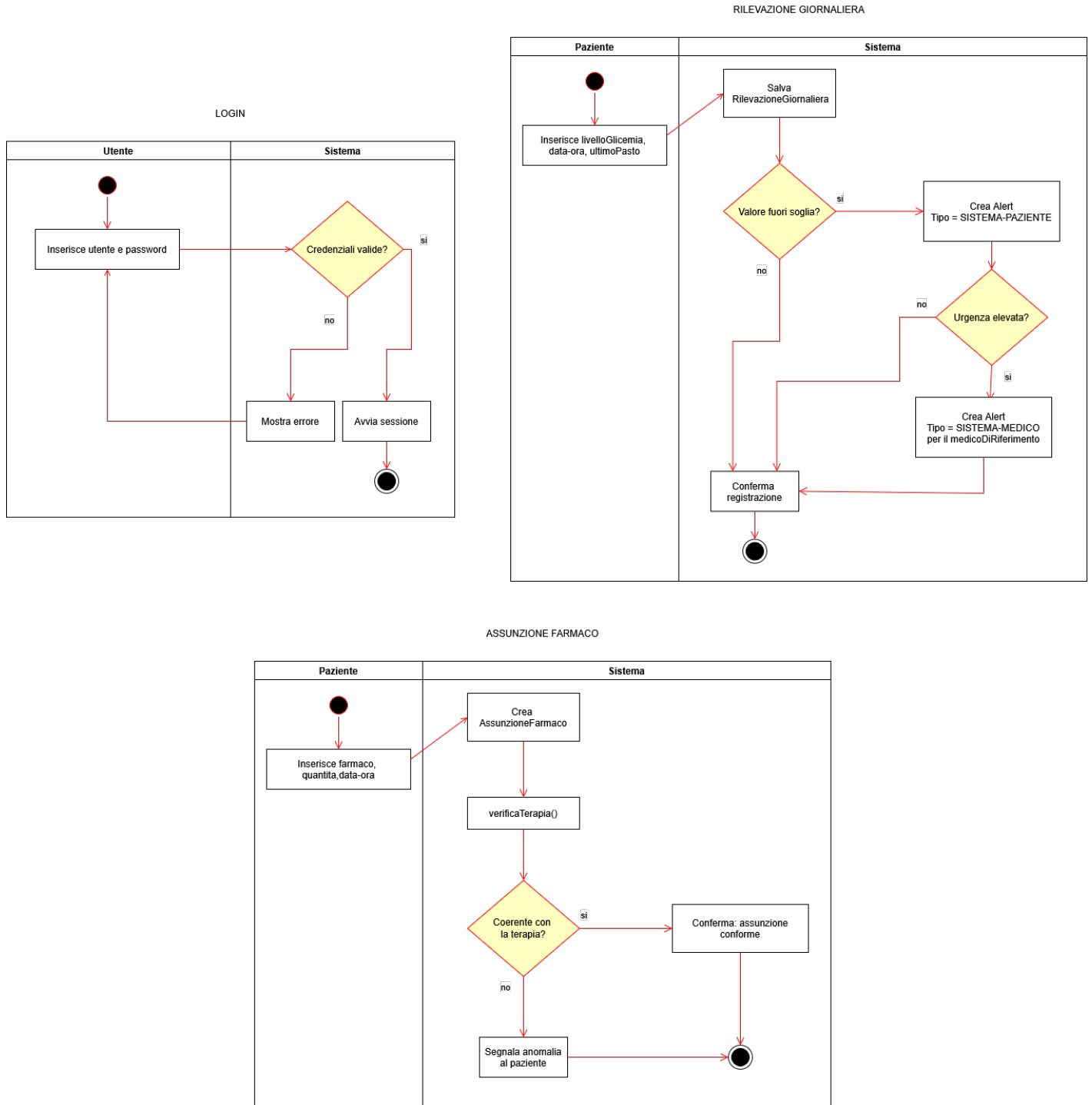
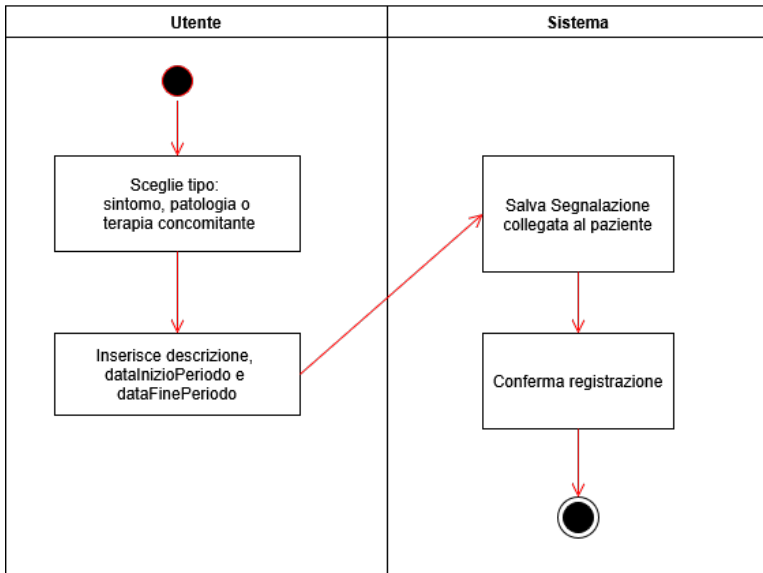
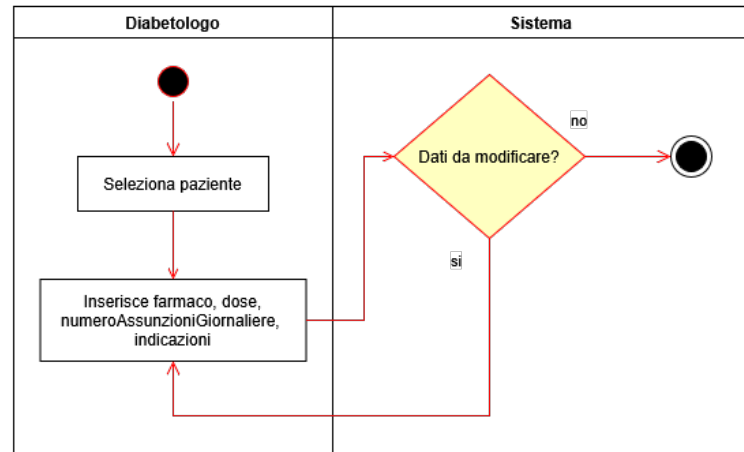


Figure 5: Activity Diagram (Parte 1)

REGISTRAZIONE SEGNALAZIONE



PRESCRIZIONE E MODIFICA DI UNA TERAPIA



AGGIORNAMENTO CARTELLA DEL PAZIENTE

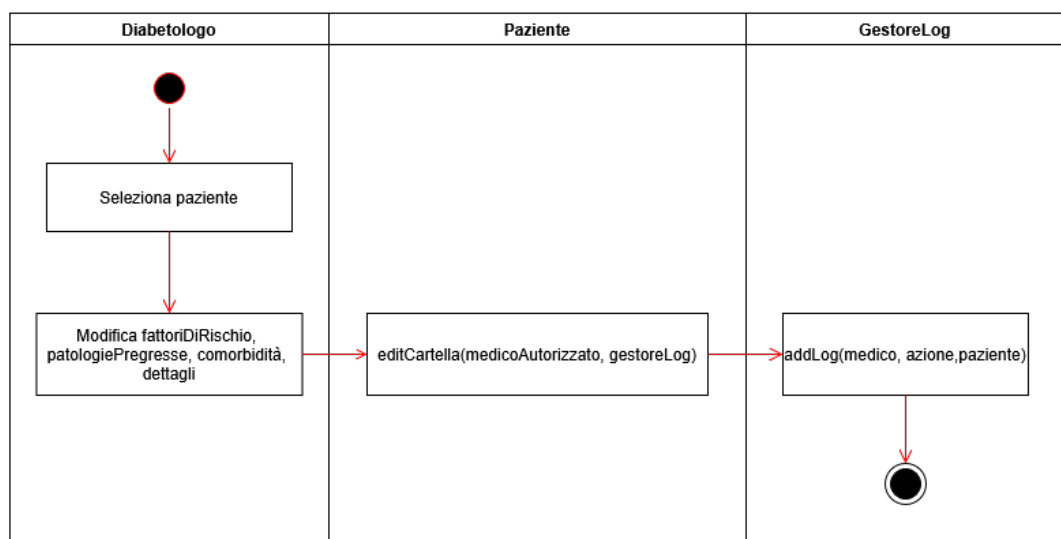


Figure 6: Activity Diagram (Parte 2)

Login: L'utente inserisce username e password. Il sistema ne verifica la validità: se risultano corrette avvia la relativa sessione e consente l'accesso alla schermata associata al ruolo dell'utente. In caso contrario viene mostrato un messaggio di errore e l'utente può digitare nuovamente le credenziali.

Rilevazione giornaliera: il paziente inserisce i dati della rilevazione, ovvero il livello di glicemia, la data e l'ora e il periodo relativo all'ultimo pasto consumato. Il sistema salva la rilevazione e controlla se il valore glicemico è fuori soglia: se lo è genera un alert per notificarlo e, se l'urgenza è elevata a causa di un alto scostamento dal normale livello, crea un altro alert destinato al medico di riferimento per segnalarlo. Al termine, il sistema conferma la registrazione della rilevazione.

Registrazione di un'assunzione e verifica della coerenza: il paziente inserisce i dati relativi al farmaco assunto, la sua quantità e la data e l'ora dell'assunzione. Il sistema salva i dati inseriti e verifica che siano coerenti con la terapia prescritta: in caso venga rilevata un'anomalia, il sistema la segnala all'utente.

Registrazione di una segnalazione: il paziente sceglie il tipo di segnalazione da effettuare e ne indica la descrizione, la data di inizio e quella di fine. Il sistema la salva e ne conferma la registrazione, associandola al paziente.

Prescrizione e modifica di una terapia: il medico seleziona il paziente al quale assegnare o modificare la terapia e inserisce i dati relativi ad essa. L'operazione viene salvata in un log. Nel caso di modifica di una terapia condivisa, la modifica viene applicata al solo paziente interessato, senza alterare la terapia degli altri pazienti associati alla precedente versione.

Aggiornamento della cartella clinica di un paziente: il diabetologo seleziona un paziente di riferimento e modifica le informazioni presenti nella sua cartella clinica. Il sistema salva le modifiche effettuate e registra l'operazione tramite un log, permettendo di mantenere la traccia dell'utente che ha effettuato l'aggiornamento.

8 Processo di sviluppo

8.1 Modelli utilizzati

Lo sviluppo del progetto è iniziato attraverso la realizzazione di un prototipo contenente la maggior parte delle funzionalità previste. L'obiettivo iniziale era definire e condividere la struttura generale del software, verificando la coerenza tra l'interfaccia grafica, l'esperienza utente, i casi d'uso analizzati e i vincoli imposti dal progetto. Il prototipo ha quindi permesso di verificare fin dalle prime fasi che fossero state considerate tutte le funzionalità necessarie. Il prototipo disponeva inoltre di una struttura software già organizzata, comprendente le classi del dominio, i controller, le viste FXML, i fogli CSS, le immagini e le altre risorse necessarie. Questa organizzazione ha costituito la base per gli sviluppi successivi, permettendo di concentrarsi sul perfezionamento delle funzionalità senza dover ridefinire la struttura generale dell'applicazione.

A partire da questa base è stato adottato un approccio di sviluppo incrementale. Il lavoro è stato suddiviso tra i membri del gruppo, sviluppando progressivamente le funzionalità relative al Responsabile, al Paziente e al Diabetologo e le operazioni che ciascun ruolo può effettuare. Successivamente sono state integrate e perfezionate ulteriori funzionalità trasversali, tra cui la gestione dei messaggi scambiati tra gli utenti e il sistema di registrazione delle modifiche effettuate sui dati dei pazienti.

La fase finale è stata dedicata al testing, con l'obiettivo di verificare il corretto funzionamento delle singole componenti e dell'applicazione nel suo complesso.

Lo sviluppo del progetto è stato quindi condotto seguendo un approccio di tipo agile, favorito dalle dimensioni ridotte del gruppo, dalla presenza di ruoli definiti e da scadenze frequenti e ravvicinate. Il lavoro è stato svolto principalmente attraverso lo sviluppo e la modifica del codice, con brevi discussioni tra i membri del gruppo prima di apportare le modifiche necessarie. Le funzionalità venivano quindi sviluppate e integrate progressivamente, per poi essere sottoposte a una revisione finale prima dell'approvazione.

8.2 Stili Architetturali

Il software segue principalmente un'architettura a strati, nella quale le responsabilità sono suddivise tra interfaccia grafica, logica applicativa e gestione della persistenza.

Il Presentation Layer è costituito dalle viste JavaFX e dai relativi controller,

che gestiscono l'interazione con l'utente e la visualizzazione dei dati.

Il Business Logic Layer comprende le classi del dominio, tra cui User, Paziente, Diabetologo, Terapia, Rilevazione, AssunzioneFarmaco, Segnalazione e le classi responsabili della gestione degli alert e delle regole applicative.

Il Data Access Layer è rappresentato dalla classe Database, che centralizza l'accesso ai dati persistenti. I controller e le altre classi che necessitano di recuperare o modificare informazioni persistenti utilizzano il Database come punto di accesso, evitando di gestire direttamente la scrittura sul file. La persistenza viene dunque realizzata tramite serializzazione Java degli oggetti all'interno del file database.data. Nel sistema è inoltre implementato un meccanismo per tracciare le modifiche effettuate sui dati dei pazienti: esse vengono infatti registrate tramite appositi log, consentendo di associare ogni modifica al diabetologo che l'ha effettuata e garantendo così la tracciabilità delle operazioni eseguite.

8.3 Design Pattern

Nel progetto è utilizzato il Design Pattern Singleton per la classe Database. Il sistema mantiene infatti una sola istanza condivisa del database, ottenuta tramite il metodo `getInstance()`. Questo permette di centralizzare l'accesso ai dati e di avere un unico punto di gestione della persistenza durante l'esecuzione dell'applicazione. Un altro utilizzo del Design Pattern Singleton è presente nella classe Session, che mantiene l'utente attualmente autenticato e permette ai diversi controller di accedere alla sessione corrente tramite un'unica istanza condivisa.

Il Database espone inoltre metodi specifici per il recupero e la modifica dei dati, ad esempio per ottenere i pazienti associati a un determinato diabetologo, le rilevazioni di un paziente o i messaggi destinati a un medico o a un paziente.

Per la gestione della GUI viene utilizzato il pattern Model-View-Controller, attraverso la struttura controller-view di JavaFX. Il pattern prevede la separazione dell'applicazione in tre componenti: il Model, che rappresenta i dati e la logica dell'applicazione, la View, che definisce l'interfaccia grafica, e il Controller, che gestisce le interazioni dell'utente coordinando le operazioni richieste. Nel progetto, il Model è rappresentato dalle classi del dominio, quali, tra le altre, User, Paziente, Diabetologo e Terapia. Le View sono invece definite tramite file FXML e descrivono la struttura e gli elementi grafici delle varie schermate, senza contenere direttamente la logica per la gestione delle operazioni. Infine, i controller, ovvero le apposite classi che gestiscono le View, gestiscono gli eventi originati dall'utente, inizializzando i componenti dell'interfaccia e coordinando le operazioni necessarie al funzionamento. I

controller interagiscono inoltre con le classi del dominio e con il database per recuperare, modificare e salvare i dati, aggiornando la View in base al risultato delle operazioni.

L'adozione di questa struttura basata sulla divisione delle responsabilità permette di separare la definizione dell'interfaccia dai dati dell'applicazione e dalla gestione degli eventi, rendendo le singole componenti più facilmente comprensibili e manutenibili.

9 Testing

9.1 Informazioni sul testing

Il testing del sistema è stato realizzato secondo un approccio White Box, con l'obiettivo di verificare il corretto comportamento delle principali componenti dell'applicazione e delle funzionalità offerte all'utente. Sono stati sviluppati test relativi sia alle classi della business logic e della persistenza dei dati, sia ai controller JavaFX. I test sono stati organizzati in Unit Test, Integration Test e System Test, in modo da verificare il sistema a diversi livelli di integrazione.

La progettazione dei test è stata effettuata considerando sia il normale utilizzo dell'applicazione sia situazioni particolari o di errore. Sono quindi stati verificati casi con dati validi, dati mancanti o incompleti, valori limite, valori fuori dalle soglie previste, elementi duplicati, elementi non presenti e ricerche senza risultati. In questo modo non è stato verificato solamente il comportamento atteso nel caso ideale, ma anche la capacità del sistema di gestire correttamente situazioni anomale o ai limiti delle condizioni previste. Per le funzionalità che modificano i dati, i test verificano inoltre l'effetto dell'operazione sia sull'interfaccia grafica sia sul database.

Ad esempio, quando viene premuto un pulsante per inserire, modificare o eliminare un elemento, viene controllato che l'interfaccia venga aggiornata correttamente e che la modifica venga effettivamente registrata nel database di test. Analogamente, nei casi in cui un'operazione non deve essere eseguita, viene verificato che i dati rimangano invariati.

Database di test e isolamento dei dati

Per rendere i test indipendenti dai dati utilizzati durante l'esecuzione dell'applicazione, è stato predisposto un database di test isolato. In questo modo i test possono inserire, modificare ed eliminare dati fittizi senza alterare il database utilizzato normalmente dall'applicazione.

La struttura del database di test viene predisposta prima dell'esecuzione dei singoli test, permettendo di partire da uno stato controllato e prevedibile. Inoltre, il riferimento al Singleton `Database` viene sostituito con l'istanza del database di test. Al termine dei test il riferimento viene ripristinato, evitando che lo stato utilizzato da un test possa influenzare l'esecuzione dei test successivi.

Modalità di test dei controller JavaFX

Per alcuni controller è presente un attributo booleano `modalitaTest`. Questa modalità viene utilizzata per adattare il comportamento del controller

durante l'esecuzione automatica dei test.

Quando `modalitaTest` è attiva, le finestre e gli `Stage` che normalmente verrebbero mostrati all'utente non vengono visualizzati a schermo. La logica interna del controller continua però ad essere eseguita normalmente. In questo modo è possibile testare autonomamente il comportamento di bottoni, campi di testo, liste, `ComboBox`, `MenuButton`, contenitori e altri elementi JavaFX, verificando il risultato delle interazioni senza aprire continuamente finestre durante l'esecuzione della suite di test.

Sincronizzazione con JavaFX tramite `runAndWait`

Le operazioni sui componenti JavaFX devono essere eseguite sul JavaFX Application Thread. Per questo motivo nei test dei controller viene utilizzato un metodo comune `runAndWait`, basato su `Platform.runLater()` e `CountDownLatch`.

Il metodo verifica innanzitutto se il codice è già in esecuzione sul JavaFX Application Thread. In caso contrario, l'operazione viene inserita nella coda di JavaFX tramite `Platform.runLater()`. Il `CountDownLatch` permette al thread del test di attendere fino al completamento dell'operazione grafica prima di continuare l'esecuzione.

Il metodo gestisce inoltre eventuali eccezioni generate durante l'esecuzione sul JavaFX Application Thread, memorizzandole e propagandole successivamente al thread del test. In questo modo eventuali errori nelle operazioni sui componenti grafici non vengono persi e possono causare il fallimento del test.

9.2 Casi di testing

9.2.1 Unit Test

I test unitari sono stati utilizzati per verificare il comportamento delle singole componenti del sistema considerate indipendentemente dalle altre funzionalità dell'applicazione. In particolare, sono state testate la gestione della sessione dell'utente e la logica relativa alla generazione degli alert.

`SessionTest`

La classe `SessionTest` verifica il corretto funzionamento della sessione dell'utente e del relativo pattern Singleton, controllando che venga mantenuta una singola istanza della sessione e che l'utente corrente venga correttamente impostato e rimosso.

- `getInstanceStessaIstanza()`: chiamata multipla di `getInstance()` per verificare che venga sempre restituita la stessa istanza della sessione.
- `funzionamentoSetELogout()`: impostazione dell'utente corrente e successivo logout per verificare che l'utente venga correttamente impostato e poi rimosso dalla sessione.

GestoreAlertTest

La classe `GestoreAlertTest` verifica la logica di generazione degli alert automatici, considerando sia la mancata assunzione dei farmaci e l'aderenza alle terapie, sia i valori glicemici rilevati prima e dopo i pasti. I test verificano inoltre le soglie associate ai diversi livelli di urgenza e impediscono la generazione ripetuta dello stesso alert.

- `verificaAssunzioniGiornaliereNonInviaPromSeTutteLeAssunzSonoFatte()`: inserimento di tutte le assunzioni previste per la giornata per verificare che non venga inviato alcun promemoria.
- `verificaAssunzioniGiornaliereInviaPromemoriaSeMancaUnaAssunzione()`: inserimento di una sola delle due assunzioni previste per verificare che venga inviato un promemoria con urgenza bassa.
- `verificaAssunzioniGiornaliereInviaPromemoriaSeNonCiSonoAssunzioni()`: verifica che, in assenza di assunzioni nella giornata, venga inviato un promemoria al paziente.
- `verificaAssunzioniGiornaliereNonInviaDueVolteLoStessoPromemoria()`: esecuzione due volte del controllo giornaliero per verificare che lo stesso promemoria non venga inviato due volte.
- `verificaAssunzioniGiornaliereIgnoraLeAssunzioniDiAltriGiorni()`: inserimento di assunzioni relative a giorni diversi per verificare che vengano considerate solo quelle della giornata corrente.
- `verificaAssunzioniGiornaliereGestiscePiuTerapieSeparatamente()`: presenza di più terapie con assunzioni diverse per verificare che venga generato un promemoria separato per ogni terapia incompleta.
- `verificaAderenzaTerapiaInviaAlertDopoTreGiorniMancati()`: verifica che dopo tre giorni consecutivi di mancata aderenza venga inviato un alert al medico con urgenza media.

- `verificaAderenzaTerapiaNonInviaAlertConMenoDiTreGiorniMancati()`: presenza di meno di tre giorni consecutivi di mancata aderenza per verificare che non venga inviato alcun alert.
- `verificaAderenzaTerapiaNonInviaAlertSeLaTerapiaECompleta()`: completamento della terapia nei tre giorni precedenti per verificare che non venga generato alcun alert.
- `verificaAderenzaTerapiaNonInviaDueVolteLoStessoAlert()`: esecuzione ripetuta del controllo di aderenza per verificare che lo stesso alert non venga inviato due volte.
- `verificaAderenzaTerapiaControllaTreGiorniDiversi()`: alternanza tra giornate complete e incomplete per verificare che il controllo consideri correttamente i tre giorni precedenti e conteggi solo quelli mancati.
- `verificaGlicemiaPrePastoSotto70InviaAlertHigh()`: rilevazione pre-pasto con glicemia pari a 69 per verificare che venga inviato un alert ad alta urgenza.
- `verificaGlicemiaPrePastoA70InviaAlertLow()`: rilevazione pre-pasto con glicemia pari a 70 per verificare che venga inviato un alert a bassa urgenza.
- `verificaGlicemiaPrePastoA79InviaAlertLow()`: rilevazione pre-pasto con glicemia pari a 79 per verificare che venga inviato un alert a bassa urgenza.
- `verificaGlicemiaPrePastoA80NonInviaAlert()`: rilevazione pre-pasto con glicemia pari a 80 per verificare che non venga inviato alcun alert.
- `verificaGlicemiaPrePastoA130NonInviaAlert()`: rilevazione pre-pasto con glicemia pari a 130 per verificare che non venga inviato alcun alert.
- `verificaGlicemiaPrePastoA131InviaAlertMedium()`: rilevazione pre-pasto con glicemia pari a 131 per verificare che venga inviato un alert a media urgenza.
- `verificaGlicemiaPrePastoA249InviaAlertMedium()`: rilevazione pre-pasto con glicemia pari a 249 per verificare che venga inviato un alert a media urgenza.
- `verificaGlicemiaPrePastoA250InviaAlertHigh()`: rilevazione pre-pasto con glicemia pari a 250 per verificare che venga inviato un alert ad alta urgenza.

- `verificaGlicemiaPostPastoSotto70InviaAlertHigh()`: rilevazione post-pasto con glicemia pari a 69 per verificare che venga inviato un alert ad alta urgenza.
- `verificaGlicemiaPostPastoA180NonInviaAlert()`: rilevazione post-pasto con glicemia pari a 180 per verificare che non venga inviato alcun alert.
- `verificaGlicemiaPostPastoA181InviaAlertMedium()`: rilevazione post-pasto con glicemia pari a 181 per verificare che venga inviato un alert a media urgenza.
- `verificaGlicemiaPostPastoA249InviaAlertMedium()`: rilevazione post-pasto con glicemia pari a 249 per verificare che venga inviato un alert a media urgenza.
- `verificaGlicemiaPostPastoA250InviaAlertHigh()`: rilevazione post-pasto con glicemia pari a 250 per verificare che venga inviato un alert ad alta urgenza.
- `verificaTuttiIPazientiControllaIPazientiDelMedico()`: verifica che il controllo di tutti i pazienti di un medico generi gli alert previsti per il suo paziente.
- `verificaTuttiIPazientiNonControllaPazientiDiUnAltroMedico()`: presenza di pazienti associati a medici diversi per verificare che vengano controllati solo quelli del medico specificato.
- `verificaTuttiIPazientiGestiscePazienteSenzaTerapie()`: presenza di un paziente senza terapie per verificare che non vengano generati alert per quel paziente.

9.2.2 Integration Test

I test di integrazione sono stati utilizzati per verificare il corretto funzionamento della persistenza e delle interazioni tra le principali classi del modello e il database. In particolare, la classe `Database` è stata testata attraverso operazioni di inserimento, ricerca, modifica, associazione ed eliminazione dei diversi elementi gestiti dall'applicazione.

DatabaseTest

La classe `DatabaseTest` verifica il comportamento del database relativamente alla gestione degli utenti, delle associazioni tra pazienti e diabetologi, delle

terapie, delle rilevazioni, delle assunzioni, delle segnalazioni, dei messaggi e dei log delle operazioni.

- `addPazienteNonAggiungeDuplicatiConStessoCodiceFiscale()`: creazione di due pazienti con lo stesso codice fiscale e aggiunta al database per verificare che non vengano inseriti duplicati.
- `addDiabetologoNonAggiungeDuplicatiConStessoCodiceFiscale()`: creazione di due diabetologi con lo stesso codice fiscale e aggiunta al database per verificare che non vengano inseriti duplicati.
- `addResponsabileNonAggiungeDuplicatiCodiceFiscaleUguale()`: creazione di due responsabili con lo stesso codice fiscale e aggiunta al database per verificare che non vengano inseriti duplicati.
- `addResponsabileNonAggiungeDuplicatiUsernameUguale()`: creazione di due responsabili con lo stesso username e aggiunta al database per verificare che non vengano inseriti duplicati.
- `getTerapieByPazienteRestituisceSoloLeTerapieDelPaziente()`: creazione di terapie associate a pazienti diversi per verificare che vengano restituite solo quelle del paziente specificato.
- `getAssunzioniByPazienteRestituisceSoloQuelleDelPaziente()`: creazione di assunzioni appartenenti a pazienti diversi per verificare che vengano restituite solo quelle del paziente specificato.
- `getRilevazioniByPazienteRestituisceSoloQuelleDelPaziente()`: creazione di rilevazioni appartenenti a pazienti diversi per verificare che vengano restituite solo quelle del paziente specificato.
- `getSegnalazioniByPazienteRestituisceSoloQuelleDelPaziente()`: creazione di segnalazioni appartenenti a pazienti diversi per verificare che vengano restituite solo quelle del paziente specificato.
- `loginRestituisceLUtenteConCredenzialiCorrette()`: inserimento di un paziente e verifica che il login restituisca l'utente con username e password corrette.
- `loginRestituisceNullConPasswordErrata()`: inserimento di un paziente e verifica che il login restituisca null con una password errata.
- `loginRestituisceNullConUtenteInesistente()`: tentativo di login con un utente inesistente per verificare che venga restituito null.

- `loginFunzionaAncheConUnResponsabile()`: inserimento di un responsabile e verifica che possa effettuare correttamente il login.
- `getMessaggioFromMedicoRitornaMessaggiDelMedico()`: inserimento di messaggi associati a diversi diabetologi per verificare che vengano restituiti solo quelli del medico specificato.
- `getMessaggioFromPazienteRitornaMessaggiDelPaziente()`: inserimento di messaggi associati a diversi pazienti per verificare che vengano restituiti solo quelli del paziente specificato.
- `pazienteVieneAssociatoAlDiabetologo()`: associazione di un paziente a un diabetologo per verificare che il paziente abbia correttamente il medico di riferimento.
- `pazientiVengonoAssociatiAlloStessoDiabetologo()`: associazione di più pazienti a uno stesso diabetologo per verificare che tutti abbiano correttamente lo stesso medico di riferimento.
- `getPazientiByDiabetologoRestituiscePazientiCorretti()`: associazione di un paziente a un diabetologo per verificare che vengano restituiti correttamente i pazienti seguiti dal medico.
- `eliminazionePazienteRimuoveIlPazienteDalDatabase()`: inserimento di un paziente e successiva eliminazione per verificare che venga rimosso dal database.
- `eliminazioneDiabetologoRimuoveIlMedicoDalDatabase()`: inserimento di un diabetologo e successiva eliminazione per verificare che venga rimosso dal database.
- `diabetologoConPazientiNonDovrebbeEssereEliminabile()`: associazione di un paziente a un diabetologo per verificare che il medico risulti ancora associato a dei pazienti e quindi non sia eliminabile.
- `usernameEsistenteRiconosceUnUsernameGiaPresente()`: inserimento di un utente e verifica che il metodo riconosca correttamente uno username già presente.
- `usernameNonEsistenteRestituisceFalse()`: verifica che uno username non presente nel database venga riconosciuto come inesistente.
- `addAssunzioneAggiungeCorrettaAssunzioneAlDatabase()`: inserimento di un'assunzione di farmaco e verifica che venga salvata correttamente con tutti i relativi dati.

- `updateAssunzioneSostituisceLElementoNellaLista()`: inserimento di un'assunzione e successiva modifica per verificare che quella originale venga sostituita con i nuovi dati.
- `updateAssunzioneNonFaNienteSeLElementoNonEPresente()`: tentativo di modificare un'assunzione non presente per verificare che il database rimanga invariato.
- `addRilevazioneAggiungeCorrettaRilevazioneAlDatabase()`: inserimento di una rilevazione e verifica che venga salvata correttamente con tutti i relativi dati.
- `updateRilevazioneSostituisceLElementoNellaLista()`: inserimento di una rilevazione e successiva modifica per verificare che quella originale venga sostituita con i nuovi dati.
- `updateRilevazioneNonFaNienteSeLElementoNonEPresente()`: tentativo di modificare una rilevazione non presente per verificare che il database rimanga invariato.
- `addSegnalazioneAggiungeCorrettaSegnalazioneAlDatabase()`: inserimento di una segnalazione e verifica che venga salvata correttamente con tutti i relativi dati.
- `updateSegnalazioneSostituisceLElementoNellaLista()`: inserimento di una segnalazione e successiva modifica per verificare che quella originale venga sostituita con i nuovi dati.
- `updateSegnalazioneNonFaNienteSeLElementoNonEPresente()`: tentativo di modificare una segnalazione non presente per verificare che il database rimanga invariato.
- `deleteAssunzioneRimuoveCorrettaAssunzioneDalDatabase()`: inserimento e successiva eliminazione di un'assunzione per verificare che venga rimossa dal database.
- `deleteRilevazioneRimuoveCorrettaRilevazioneDalDatabase()`: inserimento e successiva eliminazione di una rilevazione per verificare che venga rimossa dal database.
- `deleteSegnalazioneRimuoveCorrettaSegnalazioneDalDatabase()`: inserimento e successiva eliminazione di una segnalazione per verificare che venga rimossa dal database.

- `addTerapiaAggiungeCorrettaTerapiaAlDatabase()`: inserimento di una terapia per verificare che venga correttamente aggiunta al database.
- `addTerapiaNonAggiungeDuplicatiConTuttiICampiUguali()`: inserimento di due terapie con gli stessi dati per verificare che non vengano inseriti duplicati.
- `modificaTerapiaPazienteSostituisceLaVecchiaTerapia()`: assegnazione di una terapia a un paziente e successiva modifica per verificare che la vecchia terapia venga sostituita dalla nuova.
- `modificaTerapiaPazienteConTerapiaGiaEsistenteNonCreaDuplicati()`: modifica di una terapia con una terapia già presente nel database per verificare che non venga creato un duplicato.
- `assegnaTerapiaAggiungeIlPazienteAllaTerapia()`: assegnazione di una terapia a un paziente per verificare che il paziente venga aggiunto alla terapia.
- `assegnaTerapiaNonDuplicaUnPazienteGiaAssegnato()`: assegnazione ripetuta della stessa terapia allo stesso paziente per verificare che il paziente non venga duplicato.
- `rimuoverePazienteDaTerapiaNonRimuoveLaTerapiaDalDatabase()`: rimozione di un paziente da una terapia per verificare che la terapia rimanga comunque presente nel database.
- `rimuoverePazienteDaTerapiaLasciaLaTerapiaNelDatabase()`: rimozione del paziente dalla lista della terapia per verificare che la terapia rimanga nel database senza pazienti associati.
- `removeTerapiaRimuoveLaTerapiaDalDatabase()`: inserimento e successiva rimozione di una terapia per verificare che venga eliminata dal database.
- `removeTerapiaRimuoveLaTerapiaAncheDalleTerapieDelPaziente()`: rimozione di una terapia dal database per verificare che venga eliminata anche dalle terapie associate al paziente.
- `getLogsByPazienteRestituisceSoloIlogDelPaziente()`: inserimento di log associati a pazienti diversi per verificare che vengano restituiti solo i log relativi al paziente specificato.

- `getLogsByAutoreRestituisceSoloILogDellAutore()`: inserimento di log creati da autori diversi per verificare che vengano restituiti solo i log relativi all'autore specificato.
- `updatePazienteAggiornaLeInformazioniPersonali()`: modifica delle informazioni personali, dei fattori di rischio e degli altri dati del paziente per verificare che le nuove informazioni vengano correttamente aggiornate nel database.

9.2.3 System Test

I test di sistema sono stati utilizzati per verificare il comportamento delle funzionalità dell'applicazione dal punto di vista dell'interazione tra interfaccia grafica, controller, modello e database. I test sono stati organizzati in base alle principali aree funzionali dell'applicazione.

Diabetologo

In questa parte sono state verificate le funzionalità disponibili al diabetologo, comprendenti la visualizzazione e la ricerca dei pazienti, la consultazione dei relativi dati e dell'andamento glicemico, la modifica delle informazioni del paziente e la gestione delle terapie assegnate ai pazienti.

DiabetologoControllerTest

La classe `DiabetologoControllerTest` verifica il comportamento della schermata principale del diabetologo, la ricerca e visualizzazione dei pazienti associati e l'apertura delle funzionalità relative all'andamento glicemico e alle informazioni del paziente.

- `aperturaDiabetologoCaricaCorrettaLaSchermata()`: caricamento della schermata del diabetologo per verificare che controller, barra di ricerca e container dei pazienti vengano inizializzati correttamente.
- `ricercaPerNomeMostraSoloIlPazienteCorrispondente()`: inserimento di due pazienti e ricerca tramite nome per verificare che venga visualizzato solo il paziente corrispondente.
- `ricercaPerCognomeMostraSoloIlPazienteCorrispondente()`: inserimento di due pazienti e ricerca tramite cognome per verificare che venga visualizzato solo il paziente corrispondente.

- `ricercaPerCodiceFiscaleMostraIlPazienteCorrispondente()`: inserimento di due pazienti e ricerca tramite codice fiscale per verificare che venga visualizzato il paziente corretto.
- `ricercaNonFaDifferenzaTraMaiuscoleEMinuscole()`: ricerca di un paziente usando maiuscole e minuscole miste per verificare che la ricerca non sia case-sensitive.
- `ricercaSenzaRisultatiLasciaVuotoIlContainer()`: ricerca di un paziente inesistente per verificare che il container rimanga vuoto.
- `modificaDelTestoDiRicercaAggiornaAutomaticamenteLaLista()`: modifica consecutiva del testo di ricerca per verificare che la lista dei pazienti venga aggiornata automaticamente.
- `listaVisualizzaSoloIPazientiSeguitiDalMedico()`: inserimento di pazienti seguiti da medici diversi per verificare che il diabetologo visualizzi solo i propri pazienti.
- `boxPazienteContieneAvatarNomeEIBottoni()`: visualizzazione di un paziente per verificare che il relativo box contenga avatar, nome e pulsanti ‘Andamento’, ‘Terapia’ e ‘Info Paziente’.
- `andamentoRiceveIlPazienteCorretto()`: creazione di due pazienti con rilevazioni diverse e apertura dell’andamento del primo per verificare che venga passato il paziente corretto e visualizzata la sua rilevazione.
- `andamentoVisualizzaCorrettaRilevazioneDelGiorno()`: inserimento di una rilevazione in una determinata giornata per verificare che la visualizzazione giornaliera mostri correttamente orario e valore della glicemia.
- `andamentoVisualizzaMediaSettimanaleCorretta()`: inserimento di rilevazioni durante la settimana per verificare che la visualizzazione settimanale mostri correttamente i valori medi distinti tra prima e dopo i pasti.
- `andamentoVisualizzaMediaMensileCorretta()`: inserimento di più rilevazioni nello stesso giorno del mese per verificare che la visualizzazione mensile calcoli e mostri correttamente la media.

TerapiePazienteTest

La classe `TerapiePazienteTest` verifica la gestione delle terapie del paziente, dalla visualizzazione e ricerca fino alla creazione, assegnazione, rimozione ed eliminazione delle terapie.

- `visualizzaCorrettamenteTerapiePaziente()`: creazione e assegnazione di due terapie al paziente per verificare che vengano visualizzate entrambe nello storico delle terapie.
- `ricercaTerapiaPerNomeFarmaco()`: creazione di due terapie e ricerca tramite nome del farmaco per verificare che venga visualizzata solo la terapia corrispondente.
- `pulsanteAggiungiMostraSceltaTerapia()`: pressione del pulsante “Aggiungi” per verificare che vengano visualizzate le opzioni per creare una nuova terapia o scegliere una terapia esistente.
- `pulsanteNuovaTerapiaApriFinestraCorretta()`: apertura del menu “Aggiungi” e pressione di “Nuova terapia” per verificare che il menu venga chiuso e venga creato correttamente il `TerapiaController`.
- `pulsanteTerapiaEsistenteApriFinestraCorretta()`: apertura del menu “Aggiungi” e pressione di “Terapia esistente” per verificare che il menu venga chiuso e venga creato correttamente il `TerapieEsistentiController`.
- `pulsanteSalvaInserisceTerapiaNelDatabase()`: compilazione dei dati di una nuova terapia e pressione di “Salva” per verificare che la terapia venga inserita correttamente nel database con tutti i dati.
- `pulsanteAssegnaAggiungeTerapiaAlPaziente()`: visualizzazione di una terapia disponibile e pressione di “Assegna” per verificare che la terapia venga associata al paziente.
- `pulsanteRimuoviEliminaTerapiaDalDatabase()`: visualizzazione di una terapia disponibile e pressione di “Rimuovi” per verificare che venga eliminata dal database e dalla lista visualizzata.
- `pulsanteEliminaRimuoveTerapiaDalPaziente()`: visualizzazione di una terapia già associata al paziente e pressione di “Elimina” per verificare che venga rimossa dal paziente ma rimanga presente nel database.

InfoPazienteControllerTest

La classe `InfoPazienteControllerTest` verifica la modifica delle informazioni del paziente, comprendendo i fattori di rischio, le patologie, le comorbidità e i dettagli aggiuntivi. Viene inoltre verificata la registrazione delle modifiche nello storico, il comportamento dell'operazione di annullamento e la successiva registrazione di nuove modifiche.

- `pulsanteFattoriRischioApriMenuESelezionaFattore()`: apertura del menu dei fattori di rischio e selezione del fattore Fumatore per verificare che il fattore venga correttamente selezionato e visualizzato.
- `salvataggioFattoreDiRischioAggiornaPazienteEDatabase()`: selezione del fattore Fumatore e successivo salvataggio per verificare che il fattore di rischio venga correttamente associato al paziente e salvato nel database.
- `salvataggioModificaCampiTestualiAggiornaPaziente()`: modifica dei campi relativi a patologie pregresse, comorbidità e dettagli del paziente per verificare che le nuove informazioni vengano correttamente salvate nel database.
- `salvataggioModificaFattoriECampiTestualiCreaUnSoloLog()`: modifica di più fattori di rischio e dei campi testuali del paziente per verificare che tutte le modifiche vengano salvate correttamente e che venga creato un unico log dell'operazione con le informazioni corrette sull'autore e sul paziente modificato.
- `salvataggioSenzaModificheNonCreaNuovoLog()`: salvataggio senza apportare modifiche al paziente per verificare che i dati rimangano invariati e che non venga creato un nuovo log.
- `pulsanteStoricoApriStoricoDelleModifiche()`: modifica di un'informazione del paziente e apertura dello storico per verificare che lo storico delle modifiche venga correttamente visualizzato.
- `undoDalloStoricoRipristinaStatoPrecedente()`: modifica delle informazioni del paziente e successivo annullamento dallo storico per verificare che lo stato precedente venga correttamente ripristinato e che venga registrato il relativo log di annullamento.
- `aggiornamentoDopoUndoCreaNuovaModifica()`: modifica del paziente, annullamento della modifica e successiva nuova modifica per verificare che la nuova operazione venga correttamente registrata come una nuova modifica.

Messaggi

La gestione dei messaggi è stata verificata attraverso test relativi alla visualizzazione, alla ricerca per mittente, allo stato di lettura, al livello di urgenza e alla gestione delle notifiche.

MessaggiTest

La classe `MessaggiTest` verifica la corretta visualizzazione e gestione dei messaggi ricevuti dal diabetologo, compreso il comportamento associato all'apertura dei messaggi e alle notifiche.

- `messaggiDelDiabetologoVengonoVisualizzati()`: creazione di un messaggio del paziente verso il medico per verificare che venga correttamente visualizzato nel container dei messaggi.
- `ricercaPerMittenteMostraSoloIlMessaggioCorrispondente()`: inserimento di più messaggi dello stesso mittente e ricerca per nome per verificare che vengano visualizzati i messaggi corrispondenti.
- `ricercaPerMittenteNonFaDifferenzaTraMaiuscoleEMinuscole()`: ricerca del mittente utilizzando maiuscole e minuscole diverse per verificare che la ricerca non sia case-sensitive.
- `ricercaSenzaRisultatiLasciaVuotoIlContainer()`: ricerca di un mittente inesistente per verificare che il container dei messaggi rimanga vuoto.
- `boxMessaggioContieneAvatarMittenteAnteprimaEApri()`: visualizzazione di un messaggio per verificare che il box contenga avatar, nome del mittente e pulsante "Apri".
- `messaggioNonLettoHaOpacitaCompleta()`: visualizzazione di un messaggio non letto per verificare che il box abbia opacità completa.
- `messaggioLettoHaOpacitaRidotta()`: visualizzazione di un messaggio già letto per verificare che il box abbia opacità ridotta.
- `aperturaMessaggioLoImpostaComeLetto()`: apertura di un messaggio non letto per verificare che venga impostato come letto anche nel database.
- `aperturaDiUnMessaggioGiaLettoNonEsegueNuovamenteIlCallback()`: apertura di un messaggio già letto per verificare che il callback delle notifiche non venga eseguito nuovamente.

- `aperturaDiUnMessaggioNonLettoEsegueIlCallbackNotifiche()`: apertura di un messaggio non letto per verificare che venga eseguito il callback delle notifiche.
- `messaggioMostraIlLivelloDiUrgenza()`: visualizzazione di un messaggio ad alta urgenza per verificare che il relativo livello venga mostrato correttamente.
- `aperturaMessaggioVisualizzaMittenteETestoCompleto()`: apertura di un messaggio per verificare che vengano visualizzati correttamente mittente, testo completo e livello di urgenza.

Paziente

In questa parte sono state verificate le principali funzionalità disponibili al paziente e la gestione dello storico dei dati. I test comprendono l'inserimento di rilevazioni, assunzioni e segnalazioni, l'invio di comunicazioni al diabetologo e la consultazione, modifica ed eliminazione degli elementi dello storico.

PazienteControllerTest

La classe `PazienteControllerTest` verifica l'inserimento e il salvataggio delle rilevazioni, delle assunzioni di farmaci e delle segnalazioni, considerando sia i casi validi sia i casi in cui i dati inseriti risultano incompleti. Viene inoltre verificata la corretta gestione dell'invio di email al diabetologo di riferimento.

Rilevazione

- `inserimentoRilevazioneValidaInserisceTuttiICampi()`: compilazione di una rilevazione valida per verificare che tutti i dati vengano salvati correttamente.
- `salvataggioRilevazioneAggiungeCorrettamenteAlDatabase()`: inserimento di una rilevazione valida per verificare che venga aggiunta al database.
- `salvataggioRilevazioneSenzaCampiNonInserisceNelDatabase()`: tentativo di salvare una rilevazione senza compilare i campi per verificare che non venga inserita.
- `salvataggioRilevazioneConUnCampoMancanteNonInserisceNelDatabase()`: compilazione di tutti i dati tranne il momento della rilevazione per verificare che non venga effettuato il salvataggio.

Assunzione farmaco

- `inserimentoAssunzioneValidaInserisceTuttiICampi()`: compilazione di un'assunzione valida per verificare che paziente, data, orario, quantità e terapia vengano salvati correttamente.
- `salvataggioAssunzioneAggiungeCorrettamenteAlDatabase()`: inserimento di un'assunzione valida per verificare che venga aggiunta al database.
- `salvataggioAssunzioneSenzaCampiNonInserisceNelDatabase()`: tentativo di salvare un'assunzione senza compilare i campi per verificare che non venga inserita.
- `salvataggioAssunzioneConUnCampoMancanteNonInserisceNelDatabase()`: compilazione di tutti i dati tranne la terapia per verificare che l'assunzione non venga salvata.

Segnalazione

- `inserimentoSegnalazioneValidaInserisceTuttiICampi()`: compilazione di una segnalazione valida per verificare che data, testo e paziente vengano salvati correttamente.
- `salvataggioSegnalazioneAggiungeCorrettamenteAlDatabase()`: inserimento di una segnalazione valida per verificare che venga aggiunta al database.
- `salvataggioSegnalazioneSenzaCampiNonInserisceNelDatabase()`: tentativo di salvare una segnalazione senza compilare i campi per verificare che non venga inserita.
- `salvataggioSegnalazioneConUnCampoMancanteNonInserisceNelDatabase()`: compilazione delle date senza inserire il testo per verificare che la segnalazione non venga salvata.

Email

- `emailDelDiabetologoECorretta()`: apertura della schermata di scrittura email per verificare che il destinatario sia correttamente impostato con l'indirizzo del diabetologo.
- `invioEmailInviaCorrettaAlDiabetologoDiRiferimento()`: compilazione e invio di un'email per verificare che il messaggio venga salvato con paziente, diabetologo, testo e tipo corretti.

StoricoControllerTest

La classe `StoricoControllerTest` verifica la visualizzazione e la ricerca degli elementi presenti nello storico del paziente, oltre alle operazioni di modifica e di eliminazione delle rilevazioni, delle assunzioni e delle segnalazioni.

Apertura e visualizzazione

- `aperturaStoricoCaricaCorrettaLaSchermata()`: apertura dello storico per verificare che controller, campo di ricerca e contenitore degli elementi vengano correttamente caricati.
- `storicoVisualizzaCorrettaUnaRilevazione()`: inserimento di una rilevazione nello storico per verificare che data, glicemia e orario del pasto vengano visualizzati correttamente.
- `storicoVisualizzaCorrettaUnaAssunzione()`: inserimento di un'assunzione nello storico per verificare che data e quantità vengano visualizzate correttamente.
- `storicoVisualizzaCorrettaUnaSegnalazione()`: inserimento di una segnalazione nello storico per verificare che data e testo vengano visualizzati correttamente.

Ricerca

- `ricercaDataMostraSoloGliElementiCorrispondenti()`: inserimento di rilevazioni in date diverse e ricerca per data per verificare che venga visualizzato solo l'elemento corrispondente.
- `ricercaDataSenzaCorrispondenzeNonMostraElementi()`: ricerca di una data senza elementi corrispondenti per verificare che lo storico rimanga vuoto.

Modifica

- `pulsanteModificaRilevazioneSalvaModifica()`: apertura della modifica di una rilevazione, modifica della glicemia e salvataggio per verificare che il nuovo valore venga memorizzato nel database.
- `pulsanteModificaRilevazioneSenzaSalvareMantieneValoreOriginale()`: modifica del valore della glicemia senza effettuare il salvataggio per verificare che nel database rimanga il valore originale.

- `pulsanteModificaAssunzioneSalvaModifica()`: apertura della modifica di un'assunzione, modifica della quantità e salvataggio per verificare che il nuovo valore venga memorizzato nel database.
- `pulsanteModificaAssunzioneSenzaSalvareMantieneValoreOriginale()`: modifica della quantità di un'assunzione senza effettuare il salvataggio per verificare che nel database rimanga il valore originale.
- `pulsanteModificaSegnalazioneSalvaModifica()`: apertura della modifica di una segnalazione, modifica del testo e salvataggio per verificare che il nuovo testo venga memorizzato nel database.
- `pulsanteModificaSegnalazioneSenzaSalvareMantieneValoreOriginale()`: modifica del testo di una segnalazione senza effettuare il salvataggio per verificare che nel database rimanga il testo originale.

Eliminazione

- `pulsanteEliminaRilevazioneRimuoveElementoDalDatabase()`: pressione del pulsante di eliminazione di una rilevazione per verificare che venga rimossa sia dal database sia dallo storico visualizzato.
- `pulsanteEliminaAssunzioneRimuoveElementoDalDatabase()`: pressione del pulsante di eliminazione di un'assunzione per verificare che venga rimossa sia dal database sia dallo storico visualizzato.
- `pulsanteEliminaSegnalazioneRimuoveElementoDalDatabase()`: pressione del pulsante di eliminazione di una segnalazione per verificare che venga rimossa sia dal database sia dallo storico visualizzato.

Responsabile

La parte relativa al responsabile verifica la gestione degli utenti del sistema, comprendendo la visualizzazione e la ricerca di medici e pazienti, la modifica delle credenziali, la gestione delle associazioni tra pazienti e diabetologi e l'inserimento di nuovi utenti.

ResponsabileControllerTest

La classe `ResponsabileControllerTest` verifica il comportamento della schermata del responsabile, dalla corretta inizializzazione del profilo alla visualizzazione, ricerca, modifica ed eliminazione degli utenti presenti nel sistema.

- `initializeImpostaIlRuoloResponsabile()`: inizializzazione del controller per verificare che il ruolo visualizzato sia correttamente impostato su “Responsabile”.
- `inizializzaProfiloVisualizzaNomeECognome()`: inizializzazione del profilo di un responsabile per verificare che vengano visualizzati correttamente nome, cognome e ruolo.
- `aggiornaListeVisualizzaTuttiGliUtentiQuandoLaRicercaEVuota()`: inserimento di medici e pazienti e ricerca vuota per verificare che vengano visualizzati tutti gli utenti.
- `ricercaPerNomeMostraSoloLaPersonaCorrispondente()`: ricerca tramite nome per verificare che venga visualizzata solo la persona corrispondente.
- `ricercaPerCognomeMostraLaPersonaCorrispondente()`: ricerca tramite cognome per verificare che venga visualizzata solo la persona corrispondente.
- `ricercaNonFaDifferenzaTraMaiuscoleEMinuscole()`: ricerca con lettere maiuscole e minuscole miste per verificare che non venga fatta distinzione tra maiuscole e minuscole.
- `ricercaPerNomeECognomeFunziona()`: ricerca utilizzando nome e cognome insieme per verificare che venga trovata la persona corretta.
- `ricercaSenzaRisultatiLasciaVuotiILContainer()`: ricerca di una persona inesistente per verificare che i container di medici e pazienti rimangano vuoti.
- `modificaDelTestoDiRicercaAggiornaAutomaticamenteLeListe()`: modifica del testo di ricerca per verificare che le liste vengano aggiornate automaticamente.
- `boxPersonaContieneAvatarNomeEModificaElimina()`: visualizzazione di una persona per verificare che il relativo box contenga avatar, nome, pulsante di modifica e pulsante di eliminazione.
- `pulsanteEliminaPazienteRimuoveElementoDalDatabase()`: inserimento di un paziente e pressione del pulsante di eliminazione per verificare che venga rimosso dal database e dalla lista visualizzata.

- `pulsanteEliminaDiabetologoRimuoveElementoDalDatabase()`: inserimento di un diabetologo e pressione del pulsante di eliminazione per verificare che venga rimosso dal database e dalla lista visualizzata.

ModificaCredenzialiTest

La classe `ModificaCredenzialiTest` verifica il caricamento e la modifica dei dati personali di diabetologi e pazienti, comprese le associazioni tra pazienti e medici.

- `inizializzazioneDiabetologoPrecompilaICampi()`: inizializzazione della modifica di un diabetologo per verificare che tutti i suoi dati vengano precompilati correttamente nei campi.
- `inizializzazionePazientePrecompilaICampi()`: inizializzazione della modifica di un paziente per verificare che tutti i suoi dati vengano precompilati correttamente nei campi.
- `inizializzazioneDiabetologoConfiguraCorrettaInterfaccia()`: inizializzazione di un diabetologo per verificare che l'interfaccia mostri correttamente le opzioni per l'assegnazione dei pazienti.
- `inizializzazionePazienteConfiguraCorrettaInterfaccia()`: inizializzazione di un paziente per verificare che l'interfaccia mostri correttamente l'opzione per l'assegnazione del medico.
- `inizializzazionePazientePreselezionaIlMedicoAttuale()`: inizializzazione di un paziente già associato a un medico per verificare che il medico attuale venga preselezionato.
- `inizializzazioneDiabetologoSelezionaIPazientiGiaAssociati()`: inizializzazione di un diabetologo per verificare che i pazienti già associati risultino selezionati e quelli non associati non risultino selezionati.
- `modificaCredenzialiDiabetologoAggiornaTuttiIDati()`: modifica dei dati di un diabetologo per verificare che username, password, codice fiscale, nome, cognome ed email vengano aggiornati correttamente.
- `modificaCredenzialiPazienteAggiornaTuttiIDati()`: modifica dei dati di un paziente per verificare che username, password, codice fiscale, nome, cognome ed email vengano aggiornati correttamente.
- `modificaCredenzialiPazientePermetteDiCambiareMedico()`: modifica di un paziente associandolo a un nuovo medico per verificare che il medico di riferimento venga aggiornato.

- `modificaDiabetologoAggiornaLeAssociazioni()`: modifica di un diabetologo con più pazienti selezionati per verificare che le associazioni con i pazienti vengano aggiornate correttamente.

AggiungiPersonaTest

La classe `AggiungiPersonaTest` verifica la creazione di nuovi utenti, controllando la configurazione dell'interfaccia in base al tipo di utente, il caricamento delle associazioni disponibili e i controlli sui dati obbligatori e sugli username già presenti.

- `inizializzazioneComeMedicoConfiguraCorrettaInterfaccia()`: inizializzazione della schermata per l'aggiunta di un medico per verificare che l'interfaccia mostri correttamente le opzioni per l'assegnazione dei pazienti.
- `inizializzazioneComePazienteConfiguraCorrettaInterfaccia()`: inizializzazione della schermata per l'aggiunta di un paziente per verificare che l'interfaccia mostri correttamente l'opzione per l'assegnazione del medico.
- `inizializzazionePazienteCaricaIMediciNelComboBox()`: inserimento di più diabetologi per verificare che vengano caricati correttamente nel ComboBox dei medici.
- `inizializzazioneMedicoCaricaIPazientiNelMenuButton()`: inserimento di più pazienti per verificare che vengano caricati correttamente nel MenuButton e rappresentati tramite checkbox.
- `checkboxPazienteContieneIlPazienteComeUserData()`: creazione di un paziente e inizializzazione della schermata per verificare che la checkbox contenga correttamente nome, paziente associato come UserData e stato iniziale non selezionato.
- `handleConfermaInserisceUnNuovoMedico()`: compilazione dei dati di un nuovo medico e conferma per verificare che venga correttamente inserito nel database.
- `handleConfermaMedicoAssociaIPazientiSelezionati()`: creazione di un nuovo medico con un paziente selezionato per verificare che il paziente venga correttamente associato al medico.

- `handleConfermaInserisceUnNuovoPaziente()`: compilazione dei dati di un nuovo paziente con un medico selezionato per verificare che il paziente venga correttamente inserito e associato al medico.
- `handleConfermaNonInserisceSeMancaUnCampo()`: compilazione incompleta dei dati per verificare che un nuovo utente non venga inserito se manca un campo obbligatorio.
- `handleConfermaNonInserisceUsernameGiaEsistente()`: inserimento di un utente con uno username già presente per verificare che non venga creato un duplicato.
- `handleConfermaNonInseriscePazienteSenzaMedico()`: tentativo di inserimento di un paziente senza selezionare un medico per verificare che il paziente non venga inserito.